

BHUMI — DATA LAYER TECHNICAL DESIGN

Phases 0-7: Ingestion → Refinery → Unified Knowledge Layer → BEDROCK Evidence Broker

Target hardware: RTX 4050 Laptop (6GB VRAM) + 24GB RAM · Dev tool: Claude Code (\$1,500) · Deployment: local-first → free tier

PART 0 — READ THIS FIRST

0.1 The three answers you need before anything else

Q: Is "six stores" real, or over-engineering? Six **responsibilities** are real. Six **servers** are not. For your entire prototype and most likely for your ministry pilot, run **two physical stores**:

Physical store	Serves which responsibility
One PostgreSQL 16 with <code>pgvector</code> + <code>Apache AGE</code> + native <code>tsvector</code>	Fact Ledger, Passage Index, Relation Graph, Terrain Layer (add PostGIS later), Rule Book runtime projection
Local filesystem (→ MinIO or S3 at deploy time)	Evidence Vault (the original PDFs)

Microsoft's own Azure PostgreSQL engineering blog documents combining `pgvector` and `Apache AGE` for knowledge-graph plus semantic search **in a single engine**. That's your architecture, validated by a hyperscaler. You keep the *six-responsibility* story on your slide (it's true and it's good), and you keep two things running on your laptop.

Q: "OKF by Google" — is it real? Yes, and it's *newer and more relevant than you probably realised*. Google Cloud published the **Open Knowledge Format (OKF v0.1)** in 2026 — a vendor-neutral spec for **markdown files with YAML frontmatter** packaged as knowledge bundles for AI agents. It is

unrelated to the Open Knowledge Foundation (a UK nonprofit behind CKAN — same abbreviation, different thing).

Verdict: use OKF as your Rule Book’s file format. It is exactly what you want — Git-versionable, human-readable, LLM-readable, no framework to adopt. Details in Phase 6.

Q: Why did LayoutLM fail you last time? Not a skill gap — a category error. LayoutLM (v1/v2/v3) is a **token-classification model**: it labels tokens as `HEADER`/`QUESTION`/`ANSWER` and requires you to fine-tune it on hundreds of hand-labelled pages *per document type*. It is not a document converter, it does not output a document tree, and it does not hand you bounding-box provenance in a usable structure.

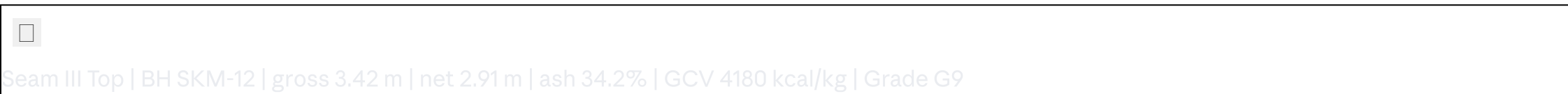
What you actually needed was a **converter**: something that ingests a PDF and emits a structured document with geometry. That’s Docling and PaddleOCR-VL. Zero-shot. No training data. Full provenance. Phase 2 covers this.

0.2 The one data example that threads through every phase

Everything below uses **one real object** so you can see the transformation end to end.

Source: `MARWATOLA_I&II_G2.pdf` — Final Geological Report, Marwatola Sector I & II Block, Sohagpur Coalfield, District Shahdol, MP, G2 stage, authored by CMPDI. Publicly downloadable from NMET.

The target row (representative of the seam-wise tables in these reports):



Watch that row become: a page image → a DoclingDocument node with a bbox → a domain-typed table → seven candidate facts → validated facts with IDs → a contextualised chunk → four graph edges → a retrievable evidence package. That’s the whole system.

0.3 MVP ladder — what “working application” means at each stage

MVP	Phases done	What a judge can <i>do</i> with it	Demo-able?
MVP- 0	Nothing user-facing.	<code>make verify</code> passes.	No

MVP	Phases done	What a judge can <i>do</i> with it	Demo-able?
0			
MVP-1	0-1-2	Upload a real GR → see the extracted document tree → click any table cell → see it highlighted on the source page	Yes — this alone is a strong demo
MVP-2	+3-4	Same, plus: tables typed as "seam table" vs "borehole schedule" → candidate facts extracted → Assay verdicts with confidence → review queue where you approve/reject	Yes — this is your novelty made visible
MVP-3	+5	Search across the corpus (hybrid), browse the knowledge graph, see clusters, ask a question and get an answer with drill-down	Yes — this is a system, not a tool
MVP-4	+6-7	BEDROCK serving three different personas with different visible tools; Rule Book edits changing system behaviour live	Yes — this is your winning demo

Design rule: every phase ends with the app still running. Never break MVP-1 to build MVP-2.

PHASE 0 — FOUNDATION & ENVIRONMENT

Layman

Before building rooms you pour a foundation. This phase installs everything, proves it all works on *your specific laptop*, and — most importantly — writes down the shape of your data so that three people don't build three incompatible versions of "what a fact looks like."

Techy

Reproducible dev environment, pinned dependencies, one Postgres with three extensions, verified GPU inference, canonical Pydantic schemas, and a

`make verify` target that fails loudly if any assumption is broken.

Modules

M0.1 — Python environment

Use `uv`, not `pip` or `conda`. It's a Rust-based installer, 10–100× faster, handles lockfiles properly, and will save you hours across a project this size.

```
bash
curl -LsSf https://astral.sh/uv/install.sh | sh
uv venv --python 3.11 # 3.11, not 3.12 — best wheel availability for ML packages
uv pip install -r requirements.lock
```

Why 3.11: PaddlePaddle, some OCR wheels, and a few graph libraries lag on 3.12/3.13. 3.11 is the current sweet spot for this stack.

M0.2 — Package manifest, with the reason for each

Package	Purpose in BHUMI	Phase used
<code>docling</code> , <code>docling-core</code>	PDF → structured document tree with bbox provenance	2
<code>paddleocr</code> / PaddleOCR-VL via llama.cpp	Hard/scanned page OCR + table recognition	2
<code>pymupdf</code> (fitz)	Page rasterisation for the drill-down viewer; fast page metadata	2, UI
<code>pydantic</code> v2	Every schema in the system. Rust core, fast validation	all
<code>sqlalchemy</code> v2 + <code>psycopg[binary]</code>	DB access layer, typed	4, 5
<code>alembic</code>	Schema migrations — non-negotiable once real data exists	0, 5
<code>pgvector</code> (python client)	Vector column type support	5

Package	Purpose in BHUMI	Phase used
<code>sentence-transformers</code>	Embeddings (<code>bge-small-en-v1.5</code> , 384-dim, ~130MB)	5
<code>rank-bm25</code> (<i>fallback only</i>)	Local BM25 if you ever need it outside Postgres	5
<code>splink</code>	Probabilistic entity resolution on a DuckDB backend	3
<code>duckdb</code>	Splink's backend; also fast local analytics on extraction metrics	3, eval
<code>spacy</code> + custom <code>EntityRuler</code>	Domain NER (seam names, borehole IDs)	3
<code>soda-core</code> <i>or</i> <code>cerberus</code>	Domain validation rules expressed as YAML data	4
<code>fastapi</code> + <code>uvicorn</code>	All service APIs	5, 7
<code>mcp</code> (Anthropic Python SDK)	BEDROCK MCP server	7
<code>llama-cpp-python</code>	Local GGUF inference (contextual chunking, entailment)	5
<code>pytest</code> + <code>pytest-cov</code>	Tests, especially for Assay rules	all
<code>ruff</code> + <code>mypy</code>	Lint + type check. Set them up now; retrofitting is misery	all
<code>streamlit</code> (<i>MVP-1/2 only</i>)	Throwaway internal review UI, replaced by React later	2, 4
<code>structlog</code>	Structured JSON logs — you'll need these for the lineage story	all

Deliberately NOT installing: LangChain, LlamaIndex, Great Expectations, Neo4j, OpenSearch, Elasticsearch, Airflow, Temporal.

Why not: every one of them is a large dependency solving a problem you don't have yet. Great Expectations in particular — one documented install pulled **107 packages**, and multiple teams report abandoning it for complexity. You will hand-write ~200 lines that do exactly what you need. That's the right trade at your scale.

M0.3 — PostgreSQL with three extensions

Use Docker Compose so it's reproducible and disposable:

```
❏  
yml  
# docker-compose.yml  
services:  
  db:  
    image: apache/age:PG16_latest # Postgres 16 + Apache AGE preinstalled  
    environment:  
      POSTGRES_DB: bhumis  
      POSTGRES_PASSWORD: bhumis_dev  
    ports: ["5432:5432"]  
    volumes: ["/data/pgdata:/var/lib/postgresql/data"]  
    shm_size: 1gb # pgvector index builds are memory-hungry
```

Then in an init migration:

```
❏  
sql  
CREATE EXTENSION IF NOT EXISTS vector; -- pgvector: embeddings  
CREATE EXTENSION IF NOT EXISTS age; -- Apache AGE: Cypher graph queries  
CREATE EXTENSION IF NOT EXISTS pg_trgm; -- trigram fuzzy matching for alias lookup  
LOAD 'age'; SET search_path = ag_catalog, "$user", public;  
SELECT create_graph('bhumis_kg');  
-- native tsvector needs no extension
```

Memory tuning for your 24GB laptop (put in `postgresql.conf`):



```
shared_buffers = 2GB
work_mem = 128MB
maintenance_work_mem = 1GB    # matters for HNSW index builds
effective_cache_size = 8GB
```

Leaves ~14GB for Python, your models, and your IDE. Don't give Postgres more — you need RAM for inference.

M0.4 — GPU verification (do this first, before writing code)

Your 6GB VRAM is the binding constraint on the whole project. Verify what actually fits:

```
python
# scripts/verify_gpu.py
import torch, subprocess
assert torch.cuda.is_available(), "No CUDA — check driver"
print(torch.cuda.get_device_name(0))      # expect: RTX 4050 Laptop GPU
free, total = torch.cuda.mem_get_info()
print(f"VRAM free {free/1e9:.2f} GB / total {total/1e9:.2f} GB")
# Expect ~5.5GB usable of 6GB. Budget against 5.0GB to be safe.
```

Your VRAM budget — plan it explicitly, it's the most important number in this project:

Concurrent workload	VRAM	Notes
Docling layout (RT-DETR) + TableFormer	~1.6 GB	Layout ~1GB, TableFormer ~600MB
PaddleOCR-VL 0.9B (Q4_K_M LM ≈300MB + vision encoder)	~1.5–2 GB	The LM quantised is tiny; encoder dominates
<code>bge-small-en-v1.5</code> embeddings	~0.3 GB	384-dim, cheap
MiniCheck-Flan-T5-Large (770M) entailment	~1.5 GB	fp16
A 3B narrative LLM at Q4_K_M	~2.5 GB	Qwen2.5-3B / Llama-3.2-3B / Phi-4-mini
Peak if all loaded	~8 GB	✗ Does not fit

Therefore, the architectural consequence — and this is a real design decision, not a workaround:

Run the pipeline as sequential stages with explicit model load/unload, not as one always-on process.

This is why Phase 2 is a **batch job**, not a live service. You ingest documents overnight (or over lunch), one stage at a time, each stage loading its model, processing everything, and freeing VRAM. The *serving* path at query time only ever needs embeddings (0.3GB) + one small LLM (2.5GB) = comfortably under budget.

Write a tiny model-lifecycle helper in Phase 0 and use it everywhere:

python


```
# src/bhumi/runtime/model_slot.py
from contextlib import contextmanager
import torch, gc

@contextmanager
def model_slot(loader, name: str):
    """Load a model, yield it, then aggressively free VRAM."""
    m = loader()
    try:
        yield m
    finally:
        del m; gc.collect(); torch.cuda.empty_cache()
        free, _ = torch.cuda.mem_get_info()
        log.info("model_unloaded", model=name, vram_free_gb=round(free/1e9, 2))
```

This 12-line file is one of the highest-value things you'll write. Every model load in the codebase goes through it.

M0.5 — Canonical schemas (the contract between everything)

These live in `src/bhumi/schemas/` and **nobody changes them without telling the whole team.**



python

```
# src/bhumi/schemas/core.py

from pydantic import BaseModel, Field
from typing import Literal, Optional
from datetime import date, datetime
from decimal import Decimal


class BBox(BaseModel):
    """A rectangle on a page. The physical basis of every provenance claim."""
    page_no: int
    l: float, t: float, r: float, b: float
    coord_origin: Literal["TOPLEFT", "BOTTOMLEFT"] = "TOPLEFT"


class SourceRef(BaseModel):
    """Where something came from. Immutable once written."""
    artifact_id: str          # sha256 of the original file
    doc_id: str              # human-readable, e.g. "GR-MARWATOLA-I-II-G2"
    version: str = "v1"
    page_no: int
    element_id: str          # the DoclingDocument node ref
    bbox: Optional[BBox] = None
    table_ref: Optional[str] = None # "Table 6.2"
    cell_ref: Optional[str] = None # "row 7 / col 3"


class CandidateFact(BaseModel):
    """A number we think might be a fact. Not yet trusted."""
    candidate_id: str
    entity_raw: str          # "SKM-12" as written
    entity_id: Optional[str] = None # canonical, after Phase 3
    metric_raw: str          # "Gross Thickness (m)" as written
    metric_key: Optional[str] = None # canonical, after Phase 3
    value_raw: str          # "3.42" exactly as extracted
```

```

value: Optional[Decimal] = None
unit_raw: Optional[str] = None
unit: Optional[str] = None
period: Optional[str] = None
status: Optional[Literal["final", "provisional", "draft"]] = None
source: SourceRef
extraction_confidence: float    # 0-1, from the READ stage
domain_type: Optional[str] = None # "seam_thickness_row", set in Phase 3

```

```

class Fact(BaseModel):
    """A validated, published fact. Bitemporal. Append-only."""
    fact_id: str
    fact_identity: str    # STABLE across revisions — the revision-trace key
    entity_id: str
    metric_key: str
    value: Decimal
    unit: str
    period: str
    status: Literal["final", "provisional"]
    valid_from: date      # when true in the world
    system_from: datetime # when we recorded it
    supersedes: Optional[str] = None
    source: SourceRef
    assay_run_id: str
    approver: Optional[str] = None
    approved_on: Optional[datetime] = None
    confidence: float

```

Note `fact_identity` **versus** `fact_id`. `fact_id` is one row. `fact_identity` is "SECL raw coal production FY2023-24" as a *concept*, stable across every revision of that number. This single field is what makes your Revision Impact Trace possible later. Getting it into the schema in Phase 0 costs nothing; retrofitting it in Phase 6 costs a week.



```
bhumi/
├── Makefile          # verify, ingest, serve, test, migrate
├── docker-compose.yml
├── pyproject.toml requirements.lock
├── .env.example
├── alembic/          # migrations from day one
├── config/
│   ├── settings.py   # pydantic-settings, env-driven
│   └── models.yaml    # which model for which stage + VRAM budget
├── rulebook/         # OKF bundles (Phase 6) — Git-versioned, human-edited
├── data/
│   ├── vault/        # original PDFs, content-addressed by sha256
│   ├── ast/          # DoclingDocument JSON per doc
│   └── pgdata/        # gitignored
├── src/bhumi/
│   ├── schemas/       # THE CONTRACT — core.py, ast.py, domain.py
│   ├── runtime/       # model_slot.py, logging.py
│   ├── acquire/       # Phase 1
│   ├── read/         # Phase 2
│   ├── domain/       # Phase 3
│   ├── assay/        # Phase 4
│   ├── knowledge/     # Phase 5
│   ├── rulebook/     # Phase 6
│   └── broker/       # Phase 7 (BEDROCK)
├── ui/               # Streamlit in MVP-1/2, React from MVP-3
├── eval/
│   ├── golden/       # hand-labelled ground truth — your best asset
│   └── harness/      # metric computation
└── tests/
```

`make verify` must check: Python version, all imports, CUDA available, VRAM $\geq$ 5GB free, Postgres reachable, all three extensions loaded, AGE graph exists, migrations current, one sample PDF converts successfully. If `make verify` passes, any teammate can start work.

Concrete flow

```
❏  
$ make verify  
/ Python 3.11.9 (uv venv)  
/ 47 packages, lockfile matches  
/ CUDA 12.4, RTX 4050 Laptop GPU, 5.61 GB free  
/ Postgres 16.4 @ localhost:5432/bhumi  
/ extensions: vector 0.7.4, age 1.5.0, pg_trgm 1.6  
/ AGE graph 'bhumi_kg' exists  
/ alembic head: 0003_assay_tables  
/ smoke: sample.pdf → 4 pages, 2 tables, 118 elements in 3.2s  
READY
```

Best practices & my additions (10)

`uv` over `pip`/`conda`. Speed compounds across hundreds of installs during a build like this.

Alembic from commit one. The moment you have real extracted data, un-migrated schema changes mean re-ingesting everything.

`model_slot()` **context manager everywhere.** With 6GB, an un-freed model is a crash three stages later.

`config/models.yaml` **declares each stage's model and its VRAM budget.** Then a startup check refuses to run a stage whose budget exceeds free VRAM — you get a clear error instead of a CUDA OOM mid-batch.

Content-address the vault (`data/vault/<sha256[:2]>/<sha256>.pdf`). Free deduplication, and the filename *is* the integrity check.

`structlog` **JSON logs from day one.** Every log line carries `doc_id`, `stage`, `element_id`. When Phase 4 rejects a fact you'll want to grep the exact provenance.

Pin CUDA-dependent versions hard. `torch`, `paddlepaddle-gpu` and driver versions interact badly. Once it works, freeze it and note the working combination in the README.

Add a `--limit N` flag to every batch script. You'll run pipelines on 3 pages a hundred times before you run them on 300.

Write the golden-set directory structure now, empty. It creates the expectation that evaluation exists. Teams that add `eval/` in week four never populate it.

A `PROVENANCE.md` decision log. Every non-obvious choice (why 3.11, why AGE over Kuzu, why not LangChain) with a date and a reason. This becomes your Slide 6 references and your defence in the technical round.

How to measure this phase

Binary: `make verify` passes on a fresh clone in under 15 minutes, on a second machine. That's it. Don't over-measure setup.

PHASE 1 — SOURCE ACQUISITION

Layman

Go and get the actual documents, and lock each one so it can never silently change. Start with the ones that are easiest to process and most useful to demo, add harder ones later.

Techy

Content-addressed acquisition with a source registry recording provenance, authority rank, classification and status *before* any parsing occurs.

The incremental corpus plan — this is the strategic part

Don't try to ingest five years of everything at once. Ingest in **four waves**, ordered by *effort-to-value ratio*.

Wave 1 — MVP-1 corpus (do this week)

Source	Why first	What it gives you
<code>MARWATOLA_I&II_G2.pdf</code> (NMET)	A real CMPDI-authored Geological Report, publicly downloadable	Bounding-coordinate tables, borehole schedules, seam nomenclature, the 800m×800m grid detail
<code>MARWATOLA_III&IV_G2.pdf</code> (NMET)	Companion report, same coalfield	Tests cross-document entity resolution immediately
South of Damuda G2 proposal (NMET)	Contains real seam-wise tables: gross/net thickness, ash%, GCV, grade bands	Your primary fact-extraction target
West of Borda / Ghonsa-Parsoda G3 (NMET)	Names geophysical log parameters (Sonic, Dual Density, Natural Gamma, Caliper, SPR, Deviation)	Borehole metadata vocabulary
NMET GR submission format (G4/G3/G2)	The government's own mandated report structure	Your extraction schema and conformance rules — see Phase 3
Coal Directory of India (CCO, 2–3 years)	Clean tabular statistics, PDF + XLS of the same release	Easy wins; the PDF↔XLS pair proves your “one release, two representations” design

Six to eight documents. That is a sufficient corpus for MVP-1 through MVP-3. Resist the urge to bulk-download; a small corpus you know intimately beats a large one you've never read.

Wave 2 — MVP-2/3 additions

Provisional Coal Statistics (gives you provisional→final revision pairs, which is what your bitemporal design exists for); 2–3 subsidiary annual reports; 20–40 Lok Sabha/Rajya Sabha coal Q&A PDFs (the demand side of your gap analysis and the corpus for topic work).

Wave 3 — Hard-mode corpus

Deliberately degraded documents: print a Wave-1 GR at low DPI, re-scan it skewed, add a rotated stamp and a handwritten margin note. **This is how you test confidence routing honestly** without misrepresenting anything — label them clearly as synthetically degraded versions of real reports.

Wave 4 — Live taps (deployment only)

DigiCoal, ERP, SWCS, CMSMS/OCBIS via read-only APIs. **Design the interface now, implement never** (for the prototype). A `LiveTapAdapter` abstract class with one mock implementation costs you an hour and completes your architecture story truthfully — you say "the interface exists, the connector is a deployment task," which is accurate and credible.

Verified download sources

NMET exploration reports: `nmet.gov.in/content/exploration_reports.php` — the listing page. Individual GRs are under `nmet.gov.in/upload/uploadfiles/files/`.

NMET GR submission format: search that same upload path for `Format-for-Geological-Report-submission-for-G4-G3-and-G2.pdf`.

Coal Directory of India: `coalcontroller.gov.in` publications section.

Provisional Coal Statistics: same site.

Parliamentary Q&A: `sansad.in` and `eparlib.nic.in`.

Verify every URL yourself before demo day — government sites reorganise, and I cannot guarantee a link resolves today. Download and vault everything locally the moment you find it. Never depend on a live fetch during a demo.

Modules

M1.1 Fetcher — one function per source type (`fetch_url`, `fetch_local`). Retries, timeout, User-Agent set politely.

M1.2 Hasher & Vault — SHA-256, write to `data/vault/<prefix>/<hash>.pdf`, return `artifact_id`.

M1.3 Source Registry — the row that must exist before anything downstream runs.

M1.4 Release Linker — the mechanism that says "this PDF and this XLS are the same publication."

M1.5 Dedup — exact by hash; near-duplicate by (title, publisher, year) fuzzy match via `pg_trgm`. Expect meaningful duplication in real government archives.

Schema



python

```
class SourceRegistration(BaseModel):
    artifact_id: str          # sha256
    doc_id: str              # "GR-MARWATOLA-I-II-G2"
    title: str
    publisher: Literal["CMPDI","GSI","MECL","IBM","NCDC","CIL","STATE_DGM","CCO","MOC","PARLIAMENT"]
    doc_kind: Literal["geological_report","project_report","annual_report",
                      "statistical_return","closure_plan","eia_emp","pq_reply","format_spec"]
    authority_rank: int      # 1 = highest. drives Phase 4 conflict resolution
    status: Literal["final","provisional","draft"]
    classification: Literal["public","internal","restricted"]
    release_id: Optional[str] # links PDF+XLS of one publication
    published_on: Optional[date]
    retrieved_at: datetime
    source_url: Optional[str]
    page_count: int
    stage: Optional[Literal["G1","G2","G3","G4"]] # GRs only
    coalfield: Optional[str]
    block: Optional[str]
    retention: str = "permanent"
```

Concrete flow



```
$ python -m bhumi.acquire register --url https://nmet.gov.in/.../MARWATOLA_I&II_G2.pdf \
--publisher CMPDI --kind geological_report --authority-rank 2 \
--status final --classification public --stage G2 --coalfield Sohagpur
```

etched 8.4 MB in 6.1s

sha256 = 4f9a2c81e7...

vaulted → data/vault/4f/4f9a2c81e7....pdf

dedup → no match

registered doc_id=GR-MARWATOLA-I-II-G2 pages=147 authority_rank=2

Best practices & my additions (9)

Register before you parse. A row in `source_registry` is the precondition for every downstream stage. Enforce it with a foreign key so it's structurally impossible to have orphan extractions.

Authority rank is a *decision*, not a lookup. Write down your ranking (CCO=1 for national statistics, MoC=1 for policy, CMPDI=2 for geology, MECL=2, GSI=2, subsidiary self-report=4, internal note=5) in `PROVENANCE.md` with reasoning. A judge asking "who decided CCO outranks SECL?" gets a real answer.

The `release_id` field is subtle and important. The Coal Directory PDF and XLS are *one release, two representations* — not two independent facts. If you treat them as two sources you'll manufacture false conflicts in Phase 4.

Never fetch during a demo. Vault everything. Add `--offline` mode that refuses network access, and run your demo in it.

Store the raw HTTP response headers alongside the artifact. `Last-Modified` and `ETag` give you free change detection for Wave 4.

Add a `notes` free-text field. You'll want to record "page 14 is a fold-out map, extraction fails" and having nowhere to put it is annoying.

My addition — an acquisition manifest. A single `corpus.yaml` listing every intended source with its metadata, so `make ingest-corpus` reproduces your entire dataset from scratch. This is the difference between a project and a pile of files.

My addition — record `page_count` at registration. It's your denominator for every extraction metric later, and it lets you sanity-check that a parse didn't silently truncate.

My addition — a `stage` field on GRs. G2 versus G1 changes what resource categories are legitimate. Capturing it at acquisition means Phase 4 can validate "you claim Proved but this is a G2 report."

How to measure

100% of vaulted artifacts have a registry row and a verifiable hash. Duplicate detection rate reported. Zero network calls during `--offline` runs.

File structure after Phase 1



MVP status: still no UI. Next phase gives you MVP-1.

PHASE 2 — READ (document → AST)

Layman

Teach the computer to see a page the way a person does: this is a heading, this is a table, this cell belongs to this column — and remember *exactly where* on the page each thing was, so we can point at it later.

Techy

A three-tier confidence-routed conversion pipeline emitting a provenance-rich document tree (DoclingDocument), with per-element extraction confidence and an explicit human-review escalation path.

What LayoutLM is

What you needed

Token classification over (text, bbox, image) triples End-to-end PDF → structured document

Requires fine-tuning on labelled data per doc type Zero-shot on unseen document types

Outputs token labels (B-HEADER, I-ANSWER)

Outputs a document tree with tables reconstructed

Excellent on FUNSD/CORD-style forms

Needed for 200-page technical reports with complex tables

No table structure recognition

Multi-level merged headers must be reconstructed

Provenance is per-token, not per-element

Need a clickable rectangle per table cell

The category you actually wanted is "document converter." That's Docling, MinerU, Marker, PaddleOCR-VL.

The tier design (this is your cost/accuracy story)





Tier tools, verified

Docling (IBM, MIT license) — github.com/docling-project/docling. Layout via RT-DETR models trained on DocLayNet (the heron-101 variant reaches 78% mAP; egret-m is the fast option at ~0.024 s/image on an A100). Tables via **TableFormer**, reported at **TEDS > 91% on FinTabNet** in ACCURATE mode. Emits `DoclingDocument` — a lossless JSON document model where **every element carries** `ProvenanceItem` **with** `page_no` **and** `bbox`. Full worker footprint

~4–6GB VRAM if you load everything; run layout and tables in separate `model_slot()` calls to stay under budget. Runs CPU-only at roughly 5–6× slower — viable for overnight batches.

PaddleOCR-VL 0.9B (Baidu, **Apache 2.0**) — `github.com/PaddlePaddle/PaddleOCR`. This is the headline finding of my research and it changes your plan. Per the PaddleOCR-VL paper (arXiv:2510.14528), it achieves **92.86 overall on OmniDocBench v1.5** — surpassing MinerU2.5-1.2B at 90.67, dots.ocr-3B at 88.41, and outperforming GPT-4o and models roughly 80× its size. Table-TEDS **90.89**, Text-Edit **0.035**, Formula-CDM **91.22**. Handles **109 languages** including Devanagari. Later versions push higher (1.5 → 94.50). **Critically for you: it was merged into llama.cpp (build b8110, Feb 19 2026), and its language model at Q4_K_M is roughly 300MB.** That means frontier-competitive table extraction on your 6GB laptop.

MinerU2.5 (OpenDataLab) — 90.67 overall v1.5, strongest on formulas and CJK. Keep as a fallback if PaddleOCR-VL underperforms on *your* documents.

Marker (Datalab) — very fast, but ⚠️ **GPL-3.0 code plus a RAIL-M weight license that restricts commercial use above a revenue threshold.** For a government deployment, avoid unless legally cleared. I'd skip it.

The honest comparison to paid tools

Capability	Your open stack	Azure AI Document Intelligence / AWS Textract / Google Document AI	Palantir Foundry
Printed-doc table extraction	At or above parity (PaddleOCR-VL 92.86 v1.5)	Comparable; per-page pricing	Not primarily an OCR product
Degraded scans, skew, warping	Good but weaker (Real5-OmniDocBench shows the gap)	Better — years of preprocessing engineering	—
Handwriting	Weak	Notably better	—
Provenance geometry	Better — DoclingDocument gives structured bbox per element	Available but flatter	—
Cost at inference	₹0	Per page, forever	Enterprise licence

Capability	Your open stack	Azure AI Document Intelligence / AWS Textract / Google Document AI	Palantir Foundry
Air-gapped operation	Yes	No	On-prem possible, very expensive
Data residency for coal data	Total	Requires cloud egress	Contractual

The composition that beats each individually: Docling for structure and provenance + PaddleOCR-VL for hard pages + PyMuPDF for the fast path. Three open tools, one pipeline, and the *only* remaining gap versus paid is degraded scans and handwriting — which is exactly what Tier 4 (human review) is for. Say this on stage; it's a genuinely strong engineering position.

The AST schema, explained properly

You mentioned using "AST JSON schema" before. **DoclingDocument is the mature version of that instinct.** Its real structure:

python

DoclingDocument:

```
schema_name: "DoclingDocument"
version: str
name: str
origin: {mimetype, binary_hash, filename}
body: GroupItem          # root of the tree
furniture: GroupItem      # headers/footers, excluded from body
texts: [TextItem | SectionHeaderItem | ListItem | CodeItem | TitleItem]
tables: [TableItem]       # each wraps TableData → TableCell[]
pictures:[PictureItem]
key_value_items: [...]
groups: [GroupItem]       # structural containers
pages: {int: PageItem}    # size, image ref
```

Every item derives from `DocItem` and carries:



python

DocItem:

```
self_ref: str    # "#/texts/42" — JSON pointer identity
parent: RefItem   # tree structure
children: [RefItem]
label: DocItemLabel # "text" | "section_header" | "table" | "caption" | ...
prov: [ProvenanceItem] # ← THE CRITICAL FIELD
```



python

ProvenanceItem:

```
page_no: int
bbox: BoundingBox # (l, t, r, b) + coord_origin
charspan: (int, int)
```

And a table cell:

```

python
TableCell:
text: str
row_span: int, col_span: int
start_row_offset_idx: int; end_row_offset_idx: int
start_col_offset_idx: int; end_col_offset_idx: int
column_header: bool; row_header: bool; row_section: bool
bbox: Optional[BoundingBox]
```

Access pattern: `doc.tables[0].prov[0].bbox` and `doc.tables[0].prov[0].page_no`. That's your drill-down, straight out of the library.

On standards: there is **no formal W3C standard for "document AST with provenance."** DoclingDocument is the de-facto choice, it's MIT-licensed, and adopting it means you inherit a well-designed schema instead of inventing one. Use it as your canonical AST and store it as JSON in `data/ast/<doc_id>.json`.

Does it need domain knowledge? Yes — and here's exactly how to add it

Docling gives you *generic* structure: "this is a table with 8 columns and 34 rows." It does **not** know that it's a seam-thickness table with a two-level header where column 3 is gross thickness in metres.

Do not fine-tune the layout model. That's the LayoutLM trap again — it needs labelled data you don't have. Instead, **layer domain typing on top as Phase 3.** Keep Phase 2 purely generic and reusable. This separation is important: it means an improvement to your domain knowledge doesn't require re-running expensive extraction.

Modules

- M2.1 Page Classifier** — born-digital vs scanned vs mixed; a quality score from DPI, text-layer coverage, skew estimate, and image entropy. ~50 lines with PyMuPDF, no ML needed.
- M2.2 Tier Router** — quality score + table complexity → tier assignment. Records *why* it chose a tier.
- M2.3 Tier 1 Fast Path** — PyMuPDF text + simple table heuristics for born-digital pages.
- M2.4 Tier 2 Docling** — layout + TableFormer, inside `model_slot()`.
- M2.5 Tier 3 PaddleOCR-VL** — degraded/complex, via llama.cpp GGUF.
- M2.6 AST Normaliser** — whatever tier ran, output conforms to DoclingDocument. **Non-negotiable: one output schema regardless of tier.**
- M2.7 Confidence Scorer** — per-element confidence combining OCR token confidence, table-structure consistency checks (do row/col spans tile the grid?), and tier prior.
- M2.8 Review Escalation** — below floor → queue with bbox.
- M2.9 Page Rasteriser** — render page PNGs at 150 DPI for the drill-down viewer. Cache them; you'll need them constantly.

Concrete flow — the Marwatola table



```
$ python -m bhumi.read run --doc GR-MARWATOLA-I-II-G2 --pages 210-220

page 214: scanned, quality 0.62, tables detected 1 (multi-level header)
→ Tier 3 (PaddleOCR-VL) : quality below 0.75 AND header depth 2
loading PaddleOCR-VL ... 1.8 GB VRAM
table reconstructed: 8 cols × 34 rows, 2 header rows, 3 merged cells
mean cell confidence 0.941 ; 2 cells below floor 0.80 → review queue
unloaded, VRAM free 5.4 GB

wrote data/ast/GR-MARWATOLA-I-II-G2.json (147 pages, 3,412 elements, 28 tables)
```

The row we care about, as it now exists in the AST:



json

{

```
"self_ref": "#/tables/17",
"label": "table",
"prov": [{ "page_no": 214, "bbox": { "l": 72.4, "t": 388.1, "r": 523.6, "b": 611.2,
      "coord_origin": "TOPLEFT"}, "charspan": [0, 1842]}],
"captions": [ "#/texts/1204"],
"data": {
  "num_rows": 34, "num_cols": 8,
  "table_cells": [
    { "text": "Seam", "column_header": true, "start_row_offset_idx": 0, "end_row_offset_idx": 2,
      "start_col_offset_idx": 0, "end_col_offset_idx": 1, "row_span": 2, "col_span": 1,
      "bbox": { "l": 74.1, "t": 389.0, "r": 128.3, "b": 412.5 }},
    { "text": "Thickness (m)", "column_header": true, "start_row_offset_idx": 0,
      "start_col_offset_idx": 2, "end_col_offset_idx": 4, "col_span": 2,
      "bbox": { "l": 186.0, "t": 389.0, "r": 298.7, "b": 400.2 }},
    { "text": "Gross", "column_header": true, "start_row_offset_idx": 1,
      "start_col_offset_idx": 2, "end_col_offset_idx": 3,
      "bbox": { "l": 186.0, "t": 400.2, "r": 242.1, "b": 412.5 }},
    { "text": "Net", "column_header": true, "start_row_offset_idx": 1,
      "start_col_offset_idx": 3, "end_col_offset_idx": 4,
      "bbox": { "l": 242.1, "t": 400.2, "r": 298.7, "b": 412.5 }},
    { "text": "Seam III Top", "start_row_offset_idx": 7, "start_col_offset_idx": 0,
      "bbox": { "l": 74.1, "t": 478.3, "r": 128.3, "b": 490.1 }},
    { "text": "SKM-12", "start_row_offset_idx": 7, "start_col_offset_idx": 1,
      "bbox": { "l": 128.3, "t": 478.3, "r": 186.0, "b": 490.1 }},
    { "text": "3.42", "start_row_offset_idx": 7, "start_col_offset_idx": 2,
      "bbox": { "l": 186.0, "t": 478.3, "r": 242.1, "b": 490.1 }},
    { "text": "2.91", "start_row_offset_idx": 7, "start_col_offset_idx": 3,
      "bbox": { "l": 242.1, "t": 478.3, "r": 298.7, "b": 490.1 }}
```

]

```
}
}
```

Look carefully at what you now have. The cell `"3.42"` knows it is at row 7, column 2; that column 2's header is `"Gross"`; that `"Gross"` sits under a spanning header `"Thickness (m)"`; and that the cell occupies a specific rectangle on page 214. **The unit "m" is recoverable from the header, not guessed.** That is the entire foundation of your "we reject rather than infer" claim.

Best practices & my additions (10)

One output schema regardless of tier. If Tier 1 and Tier 3 emit different shapes, everything downstream doubles in complexity. Normalise at M2.6, always.

Persist the AST to disk as JSON. Re-running extraction is expensive; re-running Phase 3 against a cached AST is instant. This is the single biggest iteration-speed win available.

Record the routing decision and its inputs. `{"tier":3,"reason":"quality 0.62 < 0.75","header_depth":2}`. Your cost/accuracy curve is built from these records.

Table-structure consistency as a free confidence signal. Do the row/col spans tile the grid without gaps or overlaps? If not, structure recognition failed — and this is a *deterministic* check requiring no model.

Cache page rasters at 150 DPI. 147 pages is ~40MB. Your drill-down viewer becomes instant instead of re-rendering.

My addition — extract the caption and link it. DoclingDocument has a `captions` field. "Table 6.2 — Seam-wise thickness by borehole" is the single most valuable string for Phase 3's domain typing. Never drop it.

My addition — a header-provenance resolver. Write a helper `resolve_headers(table, row, col) -> list[str]` that walks up spanning headers and returns `["Thickness (m)", "Gross"]`. Every unit inference in Phase 3 depends on this. ~40 lines, enormous leverage.

My addition — a footnote-marker detector. Coal tables use `*`, `†`, `(a)` extensively, and footnotes change meaning ("*includes inferred category"). Detect the marker in a cell, link to the footnote text, carry it forward. A judge who notices you handle footnotes will assume you've read real reports — because you have.

My addition — a `--tier-override` flag. Force a page through a specific tier for A/B comparison. This is how you generate your ablation table honestly.

My addition — treat fold-out maps and rotated pages as a known, declared failure. Detect page aspect ratio anomalies and rotation, flag them, and don't pretend to extract. Declared limitations read as maturity.

How to measure Phase 2 — build this harness now

This is where your defensible accuracy number comes from.

Metric	How	Target for MVP-2
Table structure TEDS / TEDS-S	Hand-label 25 real GR tables as HTML ground truth; compute tree-edit-distance similarity. TEDS-S ignores cell content, isolating structure	Report both; don't pre-commit to a number
Cell extraction F1	Exact-match cells against ground truth	Report per tier
Header resolution accuracy	Does <code>resolve_headers()</code> return the correct header chain?	This is <i>your</i> metric, nobody else reports it
OCR CER/WER	On a degraded-scan subset	Report separately from clean pages
% routed to human	From routing records	Report honestly — it's a feature, not a failure
Cost/accuracy curve	Accuracy vs mean VRAM-seconds per page, by tier	This is your most persuasive artifact for a technical judge

Say this on stage: *"General document parsing is close to solved — the leading benchmark is saturating, and a 0.9-billion-parameter model now beats models eighty times its size. So we didn't build a better reader. Our hard problem is turning a parsed number into a defensible fact, and that has no saturated benchmark because almost nobody does it."* That reframe converts a commodity into a differentiator.

File structure after Phase 2



data/ast/GR-MARWATOLA-I-II-G2.json
data/rasters/GR-MARWATOLA-I-II-G2/page-214.png
src/bhumi/read/{classifier,router,tier1_fast,tier2_docling,tier3_paddle,
normalise,confidence,headers,footnotes,raster}.py
ui/streamlit_app.py ← MVP-1 UI
eval/golden/tables/*.html ← ground truth
eval/harness/teds.py

✓ MVP-1 IS LIVE HERE

A Streamlit app: pick a document → see pages with detected regions overlaid → click a table → see the reconstructed grid → click a cell → the source page opens with that rectangle outlined in red. **This single screen is already a strong demo, and everything after it makes it smarter rather than replacing it.**

PHASE 3 — DOMAIN TYPING & SCHEMA BINDING

Layman

Phase 2 knows "this is a table." Phase 3 knows "this is a *seam-thickness table*, column 3 is gross thickness in metres, and SKM-12 is a borehole." This is where coal-sector knowledge enters the system — and where it must be easy to improve without touching code.

Techy

A pluggable domain layer: table-type classification, field mapping to canonical metrics, entity recognition and resolution, unit and period canonicalisation, and conformance checking against the government's mandated GR format — all driven by declarative domain packs rather than hardcoded logic.

The design principle that makes this "not a demo"

Domain knowledge lives in versioned data files, not in Python. Adding a new document type or a new metric is a pull request to a YAML file, not a code change.

This is what makes the system improvable by a CMPDI officer eventually, and it's what makes your "the retiring geologist's knowledge becomes an asset" claim literally true rather than rhetorical.

Modules

M3.1 — Table Type Classifier

Three-stage cascade, cheapest first:

- 
- Caption match (regex over the linked caption)
 - "Seam-wise thickness" → seam_thickness_table [confidence 0.95]
 - "Borehole details" → borehole_schedule
 - "Bounding coordinates" → bounding_coordinates_table
 - "Proximate analysis" → coal_quality_table
 - "Resource estimate" → resource_estimate_table
 - Header signature match (set-similarity over normalised header strings)
 - {seam, gross, net, thickness} → seam_thickness_table [confidence 0.85]
 - LLM fallback (local 3B, or Claude during development)
 - prompt with caption + headers + 3 sample rows → type + confidence

Why a cascade: stages 1 and 2 are free, deterministic and cover most real cases because government reports are formulaic. The LLM only runs on the residue. This keeps your inference cost near zero and your behaviour reproducible.

M3.2 — Domain Pack (the declarative core)



yaml

```
# domain/packs/geological_report.yaml
pack: geological_report
version: 3
applies_to:
  doc_kinds: [geological_report, project_report]

table_types:
  seam_thickness_table:
    caption_patterns:
      - "seam[- ]wise thickness"
      - "thickness of seams?"
      - "seam thickness.*borehole"
    header_signature: [seam, borehole, gross, net, thickness]
    min_signature_overlap: 3
    columns:
      - match: ["seam", "seam name", "seam no"]
        role: entity
        entity_type: seam
      - match: ["borehole", "bh", "bore hole", "bh no"]
        role: entity
        entity_type: borehole
      - match: ["gross", "gross thickness"]
        role: metric
        metric_key: seam_thickness_gross
        expect_unit: m
        unit_from_header: true      # read "(m)" from the spanning header
      - match: ["net", "net thickness"]
        role: metric
        metric_key: seam_thickness_net
        expect_unit: m
      - match: ["ash", "ash %", "ash content"]
```

```
  role: metric
  metric_key: coal_ash_pct
  expect_unit: "%"
- match: ["gcv", "calorific", "kcal"]
  role: metric
  metric_key: coal_gcv
  expect_unit: kcal/kg
- match: ["grade"]
  role: metric
  metric_key: coal_grade_band
  value_type: categorical
  allowed_scales: [gcv_g1_g17, washery_w1_w6, uhv_a_g]

entity_patterns:
borehole:
  # Real series prefixes observed in Sohagpur Coalfield GRs
  regex: '\b(SKM|SGT|MSM|CSML|CSMS|BH)[- ]?(\d{1,4})([A-Z]?)\b'
  normalise: "{prefix}-{number}{suffix}"
seam:
  regex: '\bSeam[- ]?([IVXL]+|\d+)[- ]?(Top|Middle|Bottom|A|B|E)?\b'
  normalise: "Seam {roman} {position}"
coalfield:
  gazetteer: gazetteers/coalfields.txt
block:
  gazetteer: gazetteers/blocks.txt
```

Everything in that file is data. Adding a new table type is appending YAML. Improving borehole recognition is editing one regex. That is your "dynamic enough to broaden" requirement, satisfied structurally.

M3.3 — Entity Recognition & Resolution

Recognition: spaCy `EntityRuler` with patterns generated from the domain pack, plus gazetteers. No model training. Deterministic and debuggable.

Resolution (canonicalisation): two tiers.

Tier A — deterministic alias table. For your Wave-1 corpus (~15–30 entities) a hand-curated alias table gets you >90% correctness for a few hours of work:

```
sql
CREATE TABLE entity_alias (
  alias    text PRIMARY KEY,
  alias_norm text NOT NULL, -- lowercased, punctuation-stripped
  entity_id text NOT NULL REFERENCES entity(entity_id),
  source_doc text,          -- where we first saw this spelling
  confidence real DEFAULT 1.0,
  added_by text, added_on timestampz
);
CREATE INDEX ON entity_alias USING gin (alias_norm gin_trgm_ops);
```

Tier B — probabilistic, when scale demands it. **Splink** (MIT, DuckDB backend, no Spark needed) implements the Fellegi–Sunter model. It's used at genuine government scale: per the Ministry of Justice's Splink documentation and Linacre et al. (IJPDS), it links prisons, probation, and criminal and family courts under ADR UK's Data First programme; NHS England reports up to ~19% linkage-quality improvement against the Personal Demographics Service; the Australian Bureau of Statistics used it for the 2024 National Linkage Spine. Its scores are **explainable**, which matters for your audit story in a way a neural matcher isn't.

Tier C — LLM adjudication for hard pairs. This is the right use of your Claude Code budget. Research supports it: per Peeters, Steiner & Bizer (EDBT 2025, arXiv:2310.11244), zero-shot GPT-4 outperforms fine-tuned pretrained LMs on 3 of 4 e-commerce datasets and is dramatically more robust to unseen entities — beating the best transferred PLM by $\geq 8\%$ F1, and by 40–68% under distribution shift. Their earlier work found ChatGPT reaching 82.35% F1 zero-shot on a task where RoBERTa needed 2,000 training examples. Meanwhile Ditto (Li et al., PVLDB 2020) shows fine-tuned transformers gaining up to 29% F1 over prior SOTA but **degrading 36–56% F1 under distribution shift** — which is precisely your situation, since your document types shift by decade and agency.

The composed design: Splink for the auditable bulk, Claude for the ambiguous residue, human confirmation for anything that changes a published fact. Note honestly: no head-to-head LLM-versus-Splink F1 comparison exists in the literature, so present this as a reasoned composition

rather than a benchmarked one.

M3.4 — Unit & Period Canonicalisation

Unit resolution order, and **this ordering is a real design decision worth defending**:

1. Explicit unit in the cell "3.42 m" → m
2. Unit in the column header "Thickness (m)" → m ← most common in GRs
3. Unit in the spanning parent header resolve_headers()
4. Unit in the table caption or a units row
5. Rule Book default for the metric seam_thickness_gross → m
6. NONE → REJECT to review ← never guess

Step 6 is the point. A guessed unit is how a 10× error enters an official record. Feet-versus-metres genuinely matters here: India metricated around 1957–62, so the oldest NCDC and GSI reports in the archive record thicknesses in feet and inches.

Period canonicalisation: `[FY24]`, `[2023-24]`, `[2023-2024]`, `[1.4.2023 to 31.3.2024]` → `[FY2023-24]`. Geological reports also carry non-fiscal periods (`[as at 01.04.2025]`) — handle both.

M3.5 — GR Format Conformance (your novelty N7)

The NMET **Format for Geological Report submission for G4/G3/G2** is a public government document. Encode its mandated sections and tables as a checklist, then check any GR against it:

yaml

```
# domain/conformance/nmet_gr_g2.yaml
stage: G2
mandatory_sections:
- {id: bounding_coordinates, table_type: bounding_coordinates_table}
- {id: borehole_schedule, table_type: borehole_schedule}
- {id: seam_correlation, min_tables: 1}
  {id: resource_estimate, table_type: resource_estimate_table}
- {id: coal_quality, table_type: coal_quality_table}
consistency_rules:
- id: category_vs_density
  rule: "claimed_resource_category == 'Proved' implies borehole_density >= 6.0"
  message: "Proved category requires G1 detailed exploration (~400m x 400m grid,
    approx 6-7 boreholes/sq.km). This report's density suggests G2."
```

This turns BHUMI from a reader of reports into a reviewer of them — a capability nobody asked for and everybody needs, validated against the government's own standard rather than your judgment.

M3.6 — Candidate Fact Emitter

Table + domain type + column mapping → `CandidateFact[]`, each carrying its `SourceRef` with the exact cell bbox.

Concrete flow — the Marwatola row



\$ python -m bhumi.domain type --doc GR-MARWATOLA-I-II-G2 --table 17

caption: "Table 6.2 — Seam-wise thickness by borehole, Marwatola Sector I & II"

stage 1 caption match → seam_thickness_table (conf 0.95)

pack: geological_report v3

column mapping:

col 0 "Seam" → entity(seam)

col 1 "BH No" → entity(borehole)

col 2 "Gross" ← "Thickness (m)" → metric seam_thickness_gross, unit m [from header]

col 3 "Net" ← "Thickness (m)" → metric seam_thickness_net, unit m [from header]

col 4 "Ash %" → metric coal_ash_pct, unit %

col 5 "GCV(kcal/kg)" → metric coal_gcv, unit kcal/kg

col 6 "Grade" → metric coal_grade_band, categorical

col 7 "Remarks" → ignored (no role)

row 7 → 5 candidate facts

entity resolution:

"Seam III Top" → SEAM:MARWATOLA:III_TOP (regex + alias, conf 1.00)

"SKM-12" → BH:SKM-12 (regex, conf 1.00)

△ also seen as "SKM 12", "S.K.M.-12" in III&IV report

→ alias table updated, both map to BH:SKM-12

The first candidate, in full:



json


```
{
  "candidate_id": "cand_8f21a04c",
  "entity_raw": "SKM-12", "entity_id": "BH:SKM-12",
  "metric_raw": "Gross", "metric_key": "seam_thickness_gross",
  "value_raw": "3.42", "value": "3.42",
  "unit_raw": null, "unit": "m",
  "unit_source": "column_header:Thickness (m)",
  "qualifiers": {"seam": "SEAM:MARWATOLA:III_TOP"},
  "period": "as_at_2019-03-31",
  "status": "final",
  "domain_type": "seam_thickness_table",
  "source": {
    "artifact_id": "4f9a2c81e7...", "doc_id": "GR-MARWATOLA-I-II-G2",
    "page_no": 214, "table_ref": "Table 6.2", "cell_ref": "r7c2",
    "element_id": "#/tables/17",
    "bbox": {"page_no": 214, "l": 186.0, "t": 478.3, "r": 242.1, "b": 490.1}
  },
  "extraction_confidence": 0.941
}
```

Best practices & my additions (10)

Domain knowledge in YAML, never in Python. The single most important rule in this phase.

Version your domain packs (`[version: 3]`) and record which pack version produced each candidate. When you improve the pack, you know exactly which facts to re-derive.

Cascade cheap-to-expensive. Caption regex → header signature → LLM. Government documents are formulaic; exploit that.

Never infer a unit. Six-step resolution then reject. This is a claim you can make on stage and defend.

Keep every alias forever. Resolution adds a canonical layer; it never overwrites. This is what separates you from "data cleaning" — and it's the answer to that exact challenge.

My addition — `unit_source` as a first-class field. Recording *how* the unit was determined ("column_header", "rulebook_default", "explicit_in_cell") makes unit errors debuggable and makes your provenance claim complete.

My addition — a qualifier dimension. A seam thickness isn't just (entity, metric, period) — it's (borehole, seam, metric, period). Add a `qualifiers` dict now. Retrofitting a dimension into a fact table later is genuinely painful.

My addition — grade-scale typing. G1-G17 (GCV-based, post-2012), W-I to W-VI (washery, coking), and A-G (UHV, pre-2012) are **three incomparable scales**. Type the scale explicitly and make cross-scale comparison structurally impossible. India switched coal grading from UHV to GCV with effect from 1 January 2012 — **verify this date on coal.gov.in before quoting it**, since you'll be saying it to people who lived through it.

My addition — a coring flag. A non-cored borehole yields a quantity point but not a reliable *quality* point. Capture `is_cored` and mark quality facts from non-cored holes as geophysically estimated. This is the kind of detail a geologist assumes an engineering student will get wrong.

My addition — a domain-pack test suite. For each table type, one fixture table and asserted expected candidates. When you edit a pack, tests tell you what you broke. This is what makes the domain layer safely improvable — which is the whole requirement.

How to measure Phase 3

Metric	How
Table-type accuracy	Confusion matrix over hand-typed tables; report per-stage (caption / header / LLM)
Column-mapping precision	Fraction of columns mapped to the correct metric
Unit resolution correctness Your most important metric here. Also report % rejected for missing unit	
Entity resolution P/R	On a labelled alias set
Conformance detection	Does it correctly flag a GR with a missing mandatory table?
Candidate yield per table	Sanity check — a sudden drop means a pack regression

File structure after Phase 3



```
domain/packs/{geological_report,coal_directory,annual_report}.yaml
domain/gazetteers/{coalfields,blocks,subsidiaries,seams}.txt
domain/conformance/nmet_gr_{g2,g3,g4}.yaml
src/bhumi/domain/{classifier,pack_loader,columns,entities,resolve,
units,periods,conformance,emit}.py
tests/domain/test_pack_geological_report.py
```

PHASE 4 — THE ASSAY

Layman

Before a number is trusted enough to compute with or state officially, run it through a checklist. Does it have all its pieces? Does it make physical sense? Does a more authoritative source disagree? Show a human exactly what passed, what failed, and why — and let them decide the hard cases.

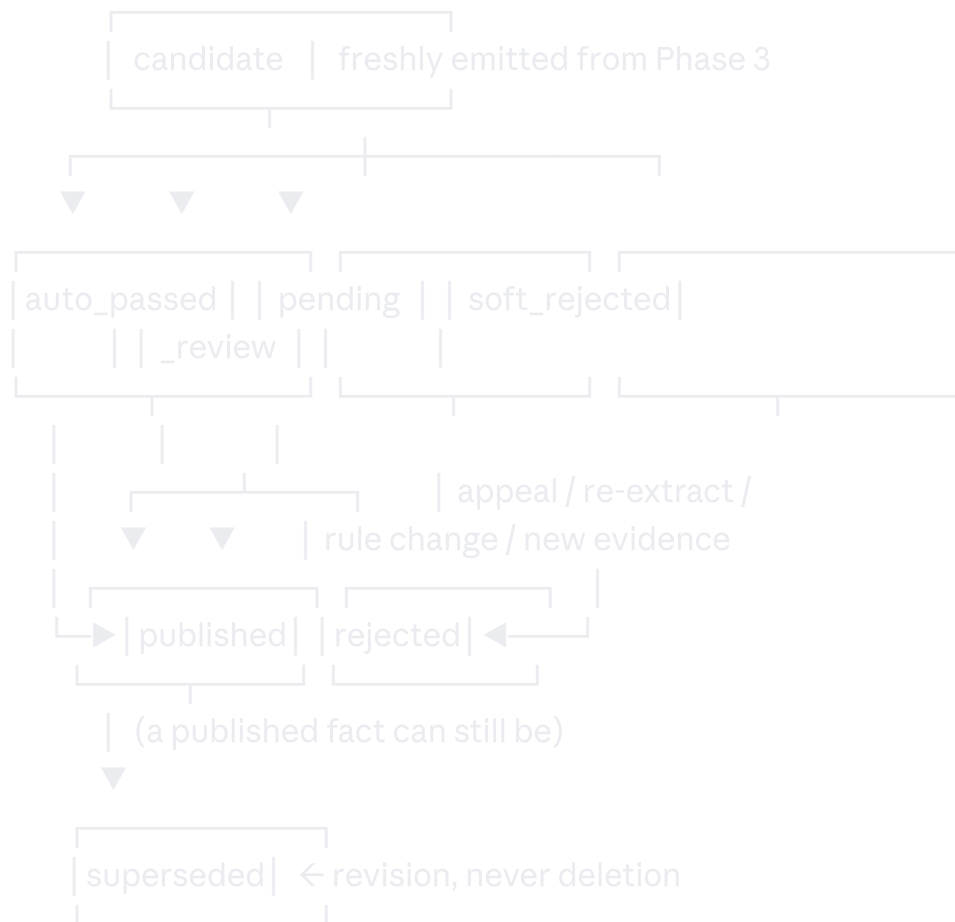
Techy

A gated fact-promotion pipeline with declarative rule evaluation, calibrated confidence, conflict resolution by source authority, and a **soft-rejection state machine** so no candidate is ever destroyed.

Your key question answered: rejected candidates that might still be needed

Never hard-delete. Use a state machine. This is the correct pattern and it also happens to be a strong story.





soft_rejected is the important state. A candidate lands there when it fails a gate but the failure is *potentially recoverable*: OCR confidence below floor, missing unit, an unresolved entity, a conflict pending adjudication. It keeps its full provenance and its failure reason, and it is **re-evaluable** when any of four things change:

A rule changes — you fix a range that was too tight → re-run the Assay over soft-rejected candidates.

The domain pack improves — a new alias resolves the entity → re-evaluate.

Re-extraction happens — the page is re-run at a higher tier → new candidate supersedes.

A human appeals — an officer says "this is correct, the rule is wrong" → adjudication, and the rule itself gets flagged for review.



sql

```
CREATE TABLE candidate_fact (  
  candidate_id text PRIMARY KEY,  
  -- ... all CandidateFact fields ...  
  state text NOT NULL CHECK (state IN  
    ('candidate','auto_passed','pending_review','soft_rejected','published','rejected')),  
  essay_run_id text NOT NULL,  
  gate_results jsonb NOT NULL, -- every gate's verdict + reason, in order  
  failed_gate text,  
  failure_code text,  
  failure_reason text,  
  reeval_count int DEFAULT 0,  
  last_reeval_at timestampz,  
  superseded_by text REFERENCES candidate_fact(candidate_id),  
  appealed_by text, appeal_note text, appeal_at timestampz  
);  
CREATE INDEX ON candidate_fact (state, failure_code); -- "show me everything soft-rejected for missing units"
```

The pitch line this earns you: *"A rejected number isn't deleted, it's parked with its reason. When we improve a rule or resolve an alias, every parked candidate is automatically re-evaluated. The system gets better retroactively over data it has already seen."* That is a genuinely strong claim, and it's a ~30-line re-evaluation job.

The seven gates, in order

Cheap deterministic checks first, human review last.



python

```
GATES = [  
    G1_fact_worthiness,    # is this a reusable quantity at all?  
    G2_shape_completeness, # entity + metric + value + unit + period + status  
    G3_metric_binding,     # binds to a Rule Book metric definition?  
    G4_authority_check,    # is this source entitled to state this?  
    G5_deterministic_rules, # range, sign, dimension, cross-field, plausibility  
    G6_conflict_detection, # does an existing fact disagree?  
    G7_risk_tiered_review, # does a human have to sign this?  
]
```

Rules as data (M4.2)

Do not use Great Expectations. One documented install pulled 107 packages and multiple teams report abandoning it for complexity. **Do not use SHACL** unless your data is RDF, which it isn't.

Use Pydantic v2 for object shape + declarative YAML for domain rules, evaluated by a small interpreter you write. For the YAML layer, `cerberus` (ISC license, zero dependencies, rules as plain dicts) or `soda-core` (Apache 2.0, YAML SodaCL checks — "if you can write a SQL WHERE clause, you can start writing Soda checks") are both good. For your scale, a ~150-line rule interpreter over your own YAML is entirely reasonable and gives you exact control over the reason strings, which matter for the review UI.



yaml

```
# rulebook/rules/geology.yaml
```

```
rules:
```

```
- id: thickness_range
```

```
  applies_to: [seam_thickness_gross, seam_thickness_net]
```

```
  check: "0 < value < 50"
```

```
  unit: m
```

```
  severity: block
```

```
  message: "Seam thickness must be between 0 and 50 m (got {value} {unit})"
```

```
- id: net_le_gross
```

```
  applies_to_pair: [seam_thickness_net, seam_thickness_gross]
```

```
  match_on: [entity_id, qualifiers.seam, period]
```

```
  check: "net.value <= gross.value"
```

```
  severity: block
```

```
  message: "Net thickness {net.value} exceeds gross {gross.value} for {entity_id}  
    / {qualifiers.seam} — physically impossible, likely an OCR digit error"
```

```
- id: no_pct_averaging
```

```
  applies_to_metric_class: percentage
```

```
  check: "aggregation != 'mean'"
```

```
  severity: block
```

```
  message: "Percentages must be computed on aggregate quantities, never averaged  
    across areas"
```

```
- id: ash_gcv_inverse
```

```
  applies_to_pair: [coal_ash_pct, coal_gcv]
```

```
  match_on: [entity_id, qualifiers.seam]
```

```
  check: "not (ash.value > 40 and gcv.value > 5500)"
```

```
  severity: warn
```

```
  message: "High ash ({ash.value}%) with high GCV ({gcv.value} kcal/kg) is  
    geologically unusual — verify"
```

- id: grade_scale_consistency
applies_to: [coal_grade_band]
check: "grade_scale_matches_coal_type(value, coal_type)"
severity: block
message: "Grade {value} is on the {detected_scale} scale but coal type is {coal_type}. G1-G17 (GCV, non-coking), W-I to W-VI (washery, coking) and A-G (UHV, pre-2012) are not interchangeable"

- id: category_vs_density
applies_to: [resource_proved_mt]
check: "borehole_density_per_sqkm >= 6.0"
severity: block
message: "Proved category requires G1 detailed exploration (~6-7 boreholes/sq.km). This block's density is {borehole_density_per_sqkm}"

Note that `ash_gcv_inverse` **is** `severity: warn`, **not** `block`. Geology has genuine exceptions. Blocking on a soft geological heuristic would make the system unusable and would rightly annoy a geologist. **Getting this severity calibration right is a judgment call only you can make** — and being able to explain *why* a given rule is warn versus block is exactly the kind of thinking a domain judge respects.

Confidence: composed, and honestly calibrated (M4.3)



python


```
def compose_confidence(c: CandidateFact, gates: list[GateResult]) -> ConfidenceReport:
    """
    Not a single opaque number. A decomposition a human can inspect.
    """
    return ConfidenceReport(
        extraction = c.extraction_confidence,    # OCR + table structure
        structural = table_grid_consistency(c),  # do spans tile the grid?
        domain_typing= c.domain_type_confidence, # table classifier
        unit      = 1.0 if c.unit_source=="explicit_in_cell"
                  else 0.9 if "header" in c.unit_source
                  else 0.6,                      # rulebook default = weaker
        entity     = c.entity_resolution_confidence,
        rules      = min(g.score for g in gates if g.severity=="warn") or 1.0,
        authority   = 1.0 / c.source_authority_rank,
        # deliberately NOT a weighted average by default:
        overall    = min(extraction, structural, unit, entity), # weakest link
    )
```

Why `min()` rather than a weighted average: a fact with perfect OCR but a guessed unit is *not* 85% trustworthy — it's as trustworthy as its weakest component. Weakest-link composition is more conservative, more defensible, and easier to explain to an officer. Say that on stage.

On calibration, be honest. Raw model confidences are not probabilities. Two options:

Temperature scaling on a held-out set, if you have enough labelled examples.

Conformal prediction — gives you distribution-free coverage guarantees (e.g. "at 0.90 threshold, $\geq 90\%$ of auto-passed facts are correct") without assuming anything about your model. **This is the more defensible choice for a small labelled set**, and it's a genuinely sophisticated thing to have in a student project.

Until you have calibrated, present confidence as a band ("high / medium / low / reject") with the decomposition visible — not as a false-precision decimal. A judge who sees "0.847" will ask what it means; a judge who sees "MEDIUM — unit inferred from Rule Book default, not stated in document" will nod.

Risk-tiered review (M4.4)



yaml

rulebook/review_policy.yaml

tiers:

- match: {destination: parliamentary_reply}
review: mandatory
reviewer_role: officer
reason: "Parliament-bound figures require an officer's approval"
- match: {metric_class: resource_estimate}
review: mandatory
reviewer_role: geologist
- match: {metric_key: clearance_status}
review: mandatory
reviewer_role: authorised_officer
- match: {confidence_band: [low, reject]}
review: mandatory
reviewer_role: any
- match: {metric_class: [publication_date, document_number]}
review: auto
- match: {domain_type: topic_assignment}
review: sampling
sample_rate: 0.05

The Review UI (M4.5) — how to *present* the assay to a human

This is your MVP-2 demo screen, and the design matters as much as the engine.



CANDIDATE cand_8f21a04c | SOURCE — page 214 |

BH:SKM-12 · Seam III Top | [rendered page image] |

seam_thickness_gross = 3.42 m |

as at 31-03-2019 · final |  ← cell outlined in red |

| || 3.42 || at the recorded bbox |

CONFIDENCE  MEDIUM |  |

extraction 0.94  |

structure 1.00  | header chain: |

unit 0.90  | Thickness (m) › Gross |

↳ from column header "Thickness(m)" |

entity 1.00  | Table 6.2, row 7, col 2 |

authority 0.50  |

↳ CMPDI rank 2 | [ prev page] [next page ] |

GATES | RELATED |

✓ G1 fact-worthy | net = 2.91 m (same row) ✓ |

✓ G2 shape complete | net ≤ gross ✓ |

✓ G3 bound → seam_thickness_gross | also in GR-MARWATOLA-III&IV: |

✓ G4 authority (CMPDI rank 2) | SKM-12 Seam III Top 3.40 m |

⚠ G5 rules: 1 warn | ⚠ 0.02 m divergence (0.6%) |

ash 34.2% + GCV 4180 → plausible | within tolerance, both retained |

✓ G6 no blocking conflict |

⌚ G7 review: geologist required | RULE BOOK |

| seam_thickness_gross |

[APPROVE] [CORRECT] [SOFT-REJECT] | "Full coal layer including dirt |

[APPEAL A RULE] | bands. Non-substitutable with |

| net thickness." v3, GM(Geology) |

Five things this screen does that a plain confidence number doesn't:

Decomposes confidence so the reviewer knows *what* is uncertain.

Shows the actual cell — verification takes four seconds, not forty minutes.

Shows the header chain, so the unit inference is auditable.

Shows related candidates, so cross-document divergence is visible at review time rather than discovered later.

Offers "Appeal a rule" — the reviewer can push back on the *system*, not just the datum. That feedback loop is what makes the Rule Book improve.

Tooling: for MVP-2, build this in Streamlit — it's throwaway and fast. Don't adopt Label Studio (community edition reportedly struggles past ~50k tasks and has no native active learning) or Prodigy (commercial). Argilla is worth a look since it integrates with ModAL for uncertainty-sampling loops, but a custom Streamlit page gives you the bespoke provenance panel above, which is the whole point.

Queue ordering: uncertainty sampling, not arrival order. Active-learning literature consistently shows this beats random selection for a fixed labelling budget — one NER study reports F1 0.91 versus 0.85 for equal effort. Your reviewer's hour should buy the most information.

Concrete flow



```
$ python -m bhumi.assay run --doc GR-MARWATOLA-I-II-G2

170 candidates from 28 tables
G1 fact-worthiness : 170 → 141 (29 narrative/non-quantitative)
G2 shape complete : 141 → 132 (9 missing unit → soft_rejected)
G3 metric binding : 132 → 130 (2 unbound metric → soft_rejected)
G4 authority : 130 → 130
G5 rules : 130 → 126 (3 range block, 1 net>gross → soft_rejected)
(7 warns recorded, not blocking)
G6 conflict : 126 → 126 (4 cross-doc divergences, all within tolerance)
G7 review tiering : 89 auto_passed · 37 pending_review

published : 89
pending_review : 37 (12 geologist · 25 low-confidence)
soft_rejected : 15 (9 unit · 2 unbound · 4 rule)
```

```
$ python -m bhumi.assay explain --candidate cand_a71f
FAILED at G5 rule 'net_le_gross'
net 3.60 m > gross 3.42 m for BH:SKM-12 / Seam III Top
→ physically impossible; likely OCR digit error (3.60 vs 2.60?)
state: soft_rejected
action: page 214 queued for Tier-3 re-extraction
```

Then, later:

```
❏
$ python -m bhumi.assay reeval --reason "domain pack v4: SKM alias variants added"
re-evaluating 15 soft_rejected candidates
4 now pass (entity resolved via new alias) → pending_review
11 still fail (unchanged reason)
```

That last command is the demo moment. The system improved retroactively over data it had already rejected.

Best practices & my additions (10)

Never hard-delete a candidate. Soft rejection with a reason code is strictly more useful and costs nothing.

Record every gate's verdict, not just the failing one. `gate_results` as JSONB. This is what makes the review panel possible and the behaviour auditable.

Gate order is cheap-to-expensive. Don't run a conflict query on a candidate that has no unit.

Weakest-link confidence, not weighted average. More conservative, easier to defend.

Separate `warn` from `block` deliberately, and document why for each rule. Blocking on soft geological heuristics makes the system unusable.

My addition — every rule carries a `message` template with interpolated values. "Seam thickness must be 0–50 m (got 61.4 m)" is actionable; "validation failed" is not. Reason quality is a product feature.

My addition — the "appeal a rule" action. It turns reviewers into Rule Book contributors and gives you a genuine human-in-the-loop improvement story rather than a claimed one.

My addition — re-evaluation as a first-class command, triggerable by rule change, pack version bump, or re-extraction. This is the single feature that most distinguishes "system" from "script."

My addition — conformal prediction for the auto-pass threshold once you have ~200 labelled examples. It gives you a defensible statement: "at this threshold, at least 90% of auto-passed facts are correct." That's a real guarantee, not a vibe.

My addition — an Assay run report written to `eval/runs/<run_id>.json`: counts per gate, per failure code, timing, VRAM. Ten runs in and you can see whether your pack changes are helping. Without it you're guessing.

How to measure Phase 4

Metric	How	Why it matters
Gate pass rates per gate	From run reports	Shows where candidates die
Auto-pass precision	Audit a random sample of auto-passed facts against ground truth	Your most important safety number

Metric	How	Why it matters
Review queue depth & mean review time	Instrumented in the UI	Feasibility of the human-in-loop design
Soft-rejection recovery rate	% of soft-rejected later published after re-eval	Proves the state machine earns its keep
Rule firing frequency	Per rule	A rule that never fires is dead; one that always fires is miscalibrated
Calibration (ECE / conformal coverage)	On the golden set	Turns confidence into a real guarantee
100% provenance completeness	Assert every published fact has a resolvable bbox	Your slide claim, mechanically verified in CI

Make that last one a test that fails the build. `test_all_published_facts_have_provenance()`. Then "100% of published facts carry page-and-cell provenance" is not a claim, it's a CI guarantee. That's the strongest form a number can take.

File structure after Phase 4

	
rulebook/rules/{geology,production,reclamation,generic}.yaml	
rulebook/review_policy.yaml	
src/bhumi/assay/{gates,rule_engine,confidence,conflict,review_queue,reeval,report}.py	
ui/pages/2_assay_review.py	
eval/runs/<run_id>.json	
tests/assay/test_rules.py	← 5 cases per rule: pass, fail, 2 edge, 1 adversarial

✓ MVP-2 IS LIVE HERE

Upload a real GR → extracted tables → domain-typed → candidate facts with decomposed confidence → review queue with the cell highlighted on the source page → approve/reject → published facts in Postgres. **This is your novelty, working and visible.**

PHASE 5 — UNIFIED KNOWLEDGE LAYER

Layman

You now have clean facts and clean document structure. This phase organises them so questions can actually be answered — numbers in a database you can add up, paragraphs in a search index you can search, relationships in a graph you can walk, and every chunk labelled well enough that it makes sense on its own.

Techy

A consolidated Postgres-centric knowledge layer providing bitemporal fact storage, hybrid lexical+vector retrieval over contextualised parent-child chunks, a multi-level property graph, and a lineage graph — all with full trace and visualisation.

5.1 Is six stores needed? The honest answer

Six responsibilities. Two physical stores. One Postgres instance.

Responsibility	MVP implementation	Migrate to (only when a threshold is crossed)
Evidence Vault	filesystem, content-addressed	MinIO / S3 + Object Lock at deployment
Passage Index	Postgres <code>tsvector</code> + <code>pgvector</code>	Meilisearch/Typesense if relevance is inadequate; OpenSearch only past ~10M chunks
Fact Ledger	Postgres tables, bitemporal	stays
Relation Graph	Apache AGE inside the same Postgres Kuzu or Neo4j if traversals dominate	
Terrain Layer	defer entirely	PostGIS when a map is genuinely needed
Rule Book	Git YAML + Postgres projection	stays

Why this is right and not lazy: Microsoft's Azure PostgreSQL team documents combining pgvector and Apache AGE for knowledge-graph plus semantic intelligence **in a single engine**. One backup, one connection string, one thing to monitor, transactional consistency across facts and graph. On a 6GB/24GB laptop, running five servers would consume the RAM you need for inference.

And it's the honest version of your slide claim. You already pitch "modular monolith first, earn the microservice premium." Live it.

Explicit migration thresholds — write these in `PROVENANCE.md`:

Vectors > ~5M or filtered-query latency > 200ms → split to Qdrant

Graph traversals > 3 hops dominating query time → Kuzu (embedded, MIT; a documented benchmark on 100K nodes/2.4M edges showed ~374× faster path queries than Neo4j — but note the original repo was archived in Oct 2025 and is now maintained as a community fork, so verify project health before committing)

Postgres FTS relevance inadequate on real queries → Meilisearch or Typesense (both added native hybrid BM25+vector search in 2025/2026, both far lighter than OpenSearch)

5.2 Fact Ledger — bitemporal, append-only




```

CREATE TABLE fact (
  fact_id      text PRIMARY KEY,
  fact_identity text NOT NULL, -- STABLE across revisions
  entity_id    text NOT NULL,
  metric_key   text NOT NULL,
  qualifiers   jsonb NOT NULL DEFAULT '{}', -- {"seam": "SEAM:....III_TOP"}
  value       numeric NOT NULL,
  unit        text NOT NULL,
  period      text NOT NULL,
  status      text NOT NULL CHECK (status IN ('final', 'provisional')),

  valid_from   date NOT NULL, -- true in the world from
  valid_to     date,          -- NULL = still valid
  system_from  timestampz NOT NULL DEFAULT now(),
  system_to    timestampz,    -- NULL = current record
  supersedes   text REFERENCES fact(fact_id),

  source_ref   jsonb NOT NULL, -- full SourceRef incl. bbox
  assay_run_id text NOT NULL,
  approver     text, approved_on timestampz,
  confidence   real NOT NULL,
  rulebook_version int NOT NULL,
  domain_pack_version int NOT NULL
);

-- fact_identity is deterministic: hash of the semantic key
-- sha256(entity_id | metric_key | canonical(qualifiers) | period)

CREATE UNIQUE INDEX fact_current ON fact (fact_identity)
WHERE system_to IS NULL; -- exactly one current version per identity

```

```
CREATE INDEX ON fact (entity_id, metric_key, period);
CREATE INDEX ON fact USING gin (qualifiers);
```

Retrieval patterns this enables:

```

sql
-- current truth
SELECT * FROM fact WHERE system_to IS NULL;
-- what we believed on 2026-06-30 (reproduce an old answer exactly)
SELECT * FROM fact WHERE system_from <= '2026-06-30'
        AND (system_to IS NULL OR system_to > '2026-06-30');
-- full revision history of one number
SELECT * FROM fact WHERE fact_identity = $1 ORDER BY system_from;
```

That second query **is** your "a June reply reproduces exactly in December" claim. One WHERE clause.

5.3 Contextual chunking — the phase that was missing

You were right that no phase covered this. Here it is properly.

The problem, in your own domain: a chunk reading

"III	Top		SKM-12		3.42		2.91		34.2		4180		G9"
------	-----	--	--------	--	------	--	------	--	------	--	------	--	-----

 is retrievable by nobody and interpretable by nothing. The numbers are orphaned from their meaning.

The technique: Anthropic's **Contextual Retrieval** — prepend LLM-generated context to each chunk *before* embedding, and index both contextual embeddings and contextual BM25. Their reported results: contextual embeddings alone cut the top-20 retrieval failure rate **5.7% → 3.7% (a 35% reduction)**; combined with contextual BM25, **5.7% → 2.9% (49%)**; adding reranking reaches **1.9% (67%)**. Indexing cost is roughly **\$1.02 per million document tokens** with prompt caching — and on your local 3B model, effectively free.

Your chunk hierarchy — parent-child, three levels:



Level 0 DOCUMENT GR-MARWATOLA-I-II-G2
Level 1 SECTION Chapter 6 — Seam Correlation and Thickness
Level 2 ELEMENT Table 6.2 (the parent — full table with headers)
Level 3 UNIT row 7 (the child — what you search)

Search the child, return the parent. Precision from the small unit, context from the large one.

The contextualisation prompt (run once at ingest, with a local 3B or Claude):



You are indexing a chunk from an Indian coal-sector Geological Report.
Write 2-3 sentences situating this chunk so it is self-contained.
Include: the document, the block/coalfield, the section, what the table reports, and the units. Do not add facts not present in the input.

DOCUMENT: Final Geological Report, Marwatola Sector I & II Block,
Sohagpur Coalfield, District Shahdol, MP. G2 stage. CMPDI, 2019.
SECTION: Chapter 6 — Seam Correlation and Thickness
TABLE: Table 6.2 — Seam-wise thickness by borehole
HEADERS: Seam | BH No | Thickness (m): Gross | Net | Ash % | GCV (kcal/kg) | Grade
CHUNK: Seam III Top | SKM-12 | 3.42 | 2.91 | 34.2 | 4180 | G9

Output, which becomes the embedded text:



From the Final Geological Report for Marwatola Sector I & II Block, Sohagpur Coalfield, District Shahdol, Madhya Pradesh (CMPDI, G2 stage, 2019), Chapter 6 on seam correlation and thickness. Table 6.2 reports seam-wise thickness by borehole, with gross and net thickness in metres, ash as a percentage, gross calorific value in kcal/kg, and grade band. This row records borehole SKM-12 intersecting Seam III Top: gross thickness 3.42 m, net thickness 2.91 m, ash 34.2%, GCV 4180 kcal/kg, grade G9.

[ORIGINAL] Seam III Top | SKM-12 | 3.42 | 2.91 | 34.2 | 4180 | G9

Now the chunk is retrievable by "seam thickness Sohagpur", "SKM-12", "grade G9 Marwatola", or "what is the net thickness of Seam III Top" — none of which would have matched the raw row.



sql

```
CREATE TABLE chunk (  
  chunk_id  text PRIMARY KEY,  
  doc_id    text NOT NULL,  
  parent_id text REFERENCES chunk(chunk_id),  
  level     int NOT NULL,          -- 0..3  
  element_ref text,              -- "#/tables/17" back into the AST  
  raw_text  text NOT NULL,  
  context_prefix text,          -- LLM-generated  
  indexed_text text NOT NULL,    -- prefix + raw, what gets embedded  
  embedding  vector(384),        -- bge-small-en-v1.5  
  tsv        tsvector GENERATED ALWAYS AS  
    (to_tsvector('english', indexed_text)) STORED,  
  source_ref jsonb NOT NULL,     -- page + bbox  
  entity_ids text[],            -- pre-extracted for filtering  
  metric_keys text[],  
  period     text,  
  classification text NOT NULL,  -- inherited from source. CRITICAL.  
  chunk_meta jsonb  
);  
  
CREATE INDEX ON chunk USING hnsw (embedding vector_cosine_ops);  
CREATE INDEX ON chunk USING gin (tsv);  
CREATE INDEX ON chunk USING gin (entity_ids);  
CREATE INDEX ON chunk (classification);  -- clearance filtering happens here
```

Hybrid retrieval with Reciprocal Rank Fusion, in one query:



sql

```

WITH lex AS (
SELECT chunk_id, ROW_NUMBER() OVER (ORDER BY ts_rank_cd(tsv, q) DESC) rnk
FROM chunk, plainto_tsquery('english', $1) q
WHERE tsv @@ q AND classification = ANY($3)  -- clearance pre-filter
LIMIT 60
),
vec AS (
SELECT chunk_id, ROW_NUMBER() OVER (ORDER BY embedding <=> $2) rnk
FROM chunk
WHERE classification = ANY($3)  -- clearance pre-filter
LIMIT 60
)
SELECT c.chunk_id, c.parent_id,
       COALESCE(1.0/(60+lex.rnk),0) + COALESCE(1.0/(60+vec.rnk),0) AS rrf
FROM chunk c
LEFT JOIN lex USING (chunk_id) LEFT JOIN vec USING (chunk_id)
WHERE lex.chunk_id IS NOT NULL OR vec.chunk_id IS NOT NULL
ORDER BY rrf DESC LIMIT 20;

```

Note where `classification = ANY($3)` **sits: inside both CTEs, before ranking.** That is "clearance applied before retrieval, not after generation," implemented. It's one clause in each subquery, and it's the difference between a security property and a slide bullet.

5.4 The knowledge graph — your hierarchy design, assessed honestly

You proposed: link documents to documents, cluster by domain (e.g. geology), connect clusters to a subsidiary, and within a document store chapters and within chapters sub-topics.

My assessment: the instinct is right, the shape needs adjusting. Here's why and how.

What's right about your plan

Multi-level is correct. A flat graph of documents cannot answer "which reports discuss Seam III in Sohagpur."

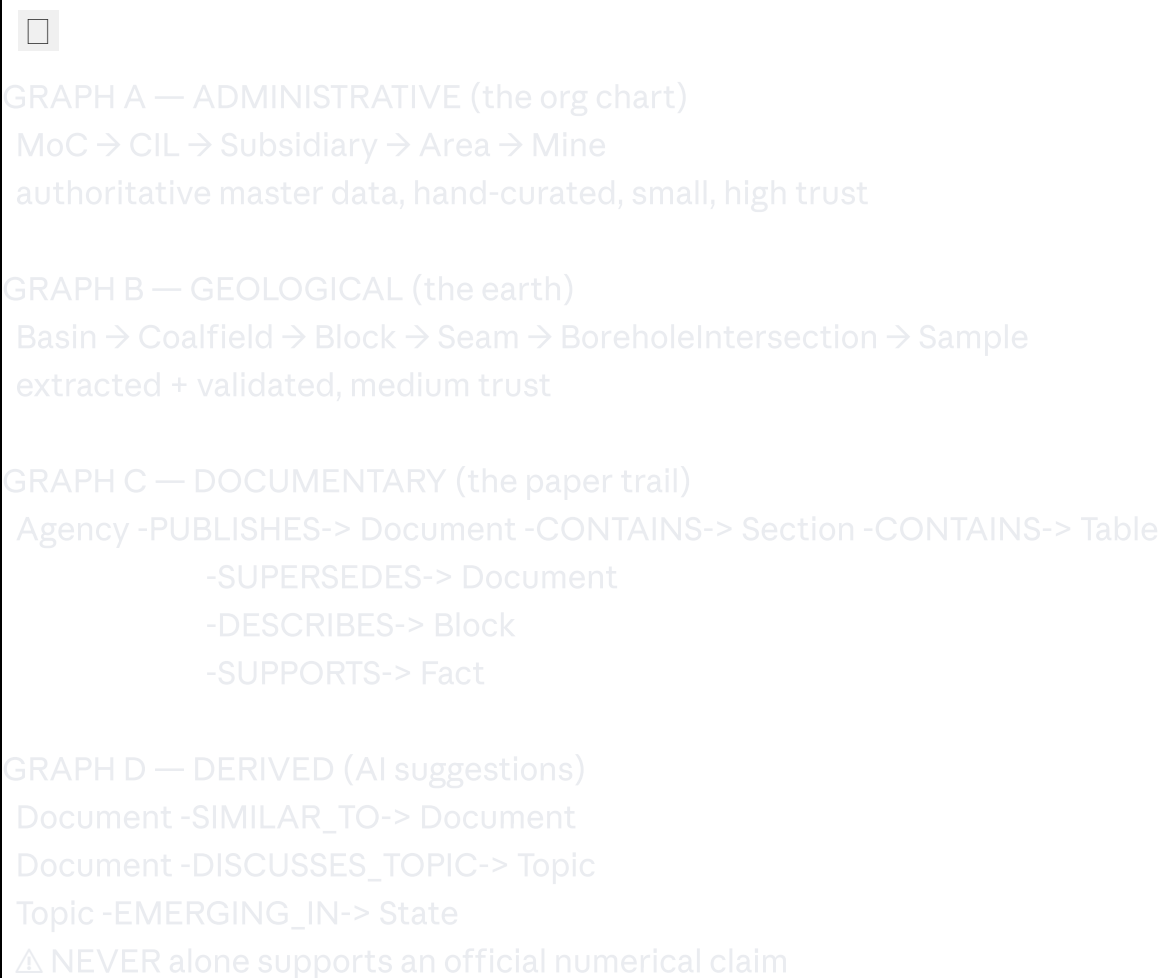
Document-to-document relationships are genuinely valuable — supersession, cross-reference, same-block.

Within-document hierarchy (chapter → subtopic) is correct and you already have it free from the AST.

What needs to change

Don't make "domain cluster" a graph node type. Domain (geology, environment, production) is an *attribute*, not a structural parent. If you model it as a node, you'll fight it constantly: a mine-closure report is both environment *and* production; a GR is geology *and* resource estimation. Model domain as a **label/property**, and use graph *queries* to get cluster views.

Separate the four graph concerns explicitly. This is the important structural point:



Why separation matters, concretely: a mine belongs administratively to a subsidiary, geographically to a district, geologically to a coalfield, and evidentially to several reports. One tree cannot express that. Four overlapping graphs can, and each has a different **trust level** — which is what lets you say “derived edges never support an official claim.”

And here's the payoff for you: `trust_layer` **is a property on every edge**. Query it and your three-trust-layer slide claim becomes a `WHERE` clause.

In Apache AGE

```

sql
SELECT * FROM cypher('bhumi_kg', $$
CREATE (:Coalfield {id:'CF:SOHAGPUR', name:'Sohagpur Coalfield', trust:'authoritative'})
CREATE (:Block {id:'BLK:MARWATOLA_I_II', name:'Marwatola Sector I & II',
          area_sqkm:21.03, stage:'G2', trust:'authoritative'})
CREATE (:Seam {id:'SEAM:MARWATOLA:III_TOP', name:'Seam III Top', trust:'validated'})
CREATE (:Borehole {id:'BH:SKM-12', series:'SKM', is_cored:true,
                  drilled_by:'CMPDI', trust:'validated'})
CREATE (:Document {id:'GR-MARWATOLA-I-II-G2', kind:'geological_report',
                  agency:'CMPDI', stage:'G2', domain:['geology','resource'],
                  trust:'authoritative'})
CREATE (:Section {id:'GR-...#ch6', title:'Seam Correlation and Thickness', level:1})
CREATE (:Table {id:'GR-...#/tables/17', ref:'Table 6.2', page:214,
               table_type:'seam_thickness_table'})
$$) AS (v agtype);
```

Edges for our one row:



sql

```
SELECT * FROM cypher('bhumikg', $$
MATCH (bh:Borehole {id:'BH:SKM-12'}), (s:Seam {id:'SEAM:MARWATOLA:III_TOP'})
CREATE (bh)-[:INTERSECTS {gross_m:3.42, net_m:2.91, fact_id:'F-000418',
trust_layer:'validated'}]->(s)
$$) AS (e agtype);
-- plus: (Block)-[:IN_COALFIELD]->(Coalfield)
-- (Seam)-[:IN_BLOCK]->(Block)
-- (Borehole)-[:DRILLED_IN]->(Block)
-- (Table)-[:IN_SECTION]->(Section)-[:IN_DOCUMENT]->(Document)
-- (Document)-[:DESCRIBES]->(Block)
-- (Table)-[:SUPPORTS]->(Fact {id:'F-000418'})
```

The query that justifies the whole graph

"Which boreholes intersect Seam III Top in Sohagpur Coalfield, what thickness did each record, and which report says so?"



```
sql
SELECT * FROM cypher('bhumikg', $$
MATCH (cf:Coalfield {id:'CF:SOHAGPUR'})<-[:IN_COALFIELD]-(b:Block)
<-[:IN_BLOCK]-(s:Seam)<-[:INTERSECTS]-(bh:Borehole),
(s)<-[:DESCRIBES_SEAM]-(t:Table)-[:IN_DOCUMENT]->(d:Document)
WHERE s.name = 'Seam III Top'
RETURN bh.id, bh.series, d.id, d.agency, t.ref
$$) AS (bh agtype, series agtype, doc agtype, agency agtype, tbl agtype);
```

Five hops. No amount of vector similarity answers this. Text search cannot join. **This query is your justification for having a graph at all** — put it in your demo, and when a judge asks "why a graph?", run it.

Tradeoffs — the honest list

Positive: multi-hop questions become tractable; supersession chains are explicit; trust layers are enforceable in query; clusters are derivable rather

than hardcoded; and the graph *is* your visualisation, which matters for your video.

Negative — say these before a judge finds them:

Graph construction is only as good as entity resolution. Bad ER produces a graph that looks impressive and answers wrongly. This is the real risk, and it's why Phase 3 matters more than Phase 5.

AGE is less mature than Neo4j. Cypher support is partial; some syntax differs. You will hit rough edges.

Maintenance cost on re-ingest. When a document is re-extracted, its extracted edges must be invalidated and rebuilt. Design this now: tag every edge with `assay_run_id` and `domain_pack_version` so you can delete-and-rebuild a document's subgraph deterministically.

Over-modelling is a real trap. Every node type you add is a type you must populate, validate and query. Start with the seven types above. Add an eighth only when a query demands it.

Derived edges are seductive. Similarity edges look great in a visualisation and mean almost nothing. Keep them visually distinct (dashed, different colour) and never let them into a numeric claim.

My recommendation on scope: build Graphs A, B and C for MVP-3. Build Graph D in the Topic Intelligence phase, clearly separated. That's a real graph, honestly scoped.

5.5 Lineage graph (the drill-down and revision-trace substrate)

Model this on **W3C PROV-O** (Entity / Activity / Agent) rather than a homemade schema — it's a real standard and citing it is worth marks.

```
sql
CREATE TABLE lineage_edge (
  from_kind text, from_id text, -- 'artifact','element','candidate','fact','chunk','claim'
  to_kind text, to_id text,
  activity text, -- 'scan','read','type','assay','chunk','compute'
  agent text, -- 'docling@2.x','paddleocr-vl@0.9b','officer:xyz'
  run_id text,
  at timestampz DEFAULT now(),
  PRIMARY KEY (from_kind, from_id, to_kind, to_id, activity)
);
```

Read backward → drill-down. Read forward → revision impact. Same table, two traversal directions. That's the "one graph, two directions" claim, and it's ~20 lines of recursive CTE.

5.6 Trace & visualisation (important for your video)

Four views. Build them; they *are* the demo.

Document explorer — page image with element overlays, click-through to the AST node.

Fact explorer — filterable table of facts, each row expanding into its full provenance chain and rendered source cell.

Graph explorer — interactive subgraph around a selected entity, edges coloured by trust layer, dashed for derived. Use `cytoscape.js` or `react-force-graph`; export a static SVG for slides.

Pipeline trace — for one document, a timeline of every stage: tier chosen, models loaded, VRAM used, candidates emitted, gates fired, facts published, chunks created, edges added. **This screen alone answers "is this a mock?" definitively.**

Best practices & my additions (10)

One Postgres, three extensions, two physical stores. Revisit only against a written threshold.

`fact_identity` **as a deterministic hash of the semantic key.** Same fact from two documents → same identity → conflict detection is a unique-index violation rather than a fuzzy comparison.

Clearance inside the retrieval CTEs, never after. One clause, and it makes the security claim literally true.

Contextual chunking is not optional in this domain. Orphaned table rows are your single biggest retrieval failure mode, and the published gain (35–67% fewer failures) is the largest single improvement available to you.

Search child, return parent. Non-negotiable for tables.

My addition — persist `context_prefix` separately from `indexed_text`. When you improve the contextualisation prompt, you re-generate prefixes without re-chunking, and you can A/B two prompts on the same chunks.

My addition — tag every graph edge with `assay_run_id` + `domain_pack_version`. Deterministic subgraph rebuild on re-ingest. Without this, re-ingestion corrupts your graph slowly and invisibly.

My addition — a `graph_invariants` test suite. No orphan seams, every borehole in exactly one block, no `INTERSECTS` edge without a supporting fact, no derived edge with `trust_layer='authoritative'`. Run in CI. Graphs decay silently; invariants catch it.

My addition — materialised views for hot queries. "Facts per entity per metric per period, current only" as a matview refreshed after each Assay run. Your UI stays fast on a laptop.

My addition — an `export_evidence_pack(fact_ids)` **function** that emits a folder of cropped source-page images plus a manifest JSON. This is a real deliverable an officer would want, it's ~50 lines, and it's an extremely strong demo artifact.

How to measure Phase 5

Metric	How
Retrieval Recall@10 with ablation	naive → +hybrid → +contextual → +parent-child → +rerank, on 60 golden questions. Build this table; it's your most persuasive technical artifact
MRR / nDCG@10	Same golden set
Chunk self-containment	Sample 30 chunks; can a human identify the entity, metric and unit from the chunk alone?
Graph invariant pass rate	CI
Multi-hop query correctness	On 10 hand-written 3+ hop questions
Bitemporal reproducibility	Snapshot an answer, revise a fact, re-query as-of the snapshot date, assert identical
p95 query latency	Must stay under 3s on your laptop

File structure after Phase 5



```
alembic/versions/*_fact_ledger.py, *_chunk.py, *_lineage.py, *_age_graph.py
src/bhumi/knowledge/
|— ledger/{write,query,bitemporal,identity}.py
|— chunking/{hierarchy,contextualise,embed,index}.py
|— retrieval/{hybrid_rrf,parent_expand,rerank,filters}.py
|— graph/{schema,build_admin,build_geo,build_doc,queries,invariants}.py
|— lineage/{write,trace_back,trace_forward}.py
|— export/evidence_pack.py
ui/pages/{3_documents,4_facts,5_graph,6_pipeline_trace}.py
eval/golden/questions.jsonl
eval/harness/{retrieval_ablation,graph_invariants}.py
```

✓ MVP-3 IS LIVE HERE

Search the corpus, browse the graph, ask a question and get an answer with full drill-down, see the pipeline trace. **This is a system.**

PHASE 6 — THE RULE BOOK (OKF)

Layman

One place where "what does this officially mean" lives — as versioned files a human can read and edit, not as knowledge in someone's head or logic buried in code.

Techy

A Git-versioned, OKF-formatted knowledge base of metric definitions, validation rules, authority rankings and terminology, compiled into a Postgres runtime projection on commit.

OKF: what it actually is, and my verdict

Google Cloud published the Open Knowledge Format (OKF v0.1) in 2026 — a vendor-neutral spec for LLM-readable knowledge bundles built from **markdown files with YAML frontmatter**, authored by Sam McVeety and Amir Hormati per the Google Cloud blog. It is **not** the Open Knowledge Foundation (the UK nonprofit behind CKAN — same abbreviation, unrelated organisation).

Verdict: use OKF as your file format. It fits your requirements almost exactly.

Why it's a good fit: markdown+YAML is human-editable by a CMPDI officer without training; Git gives you versioning, review and blame for free; an LLM can read it natively (which matters for your Narrative Track and BEDROCK); it's vendor-neutral so you're not locked in; and it's a *format*, not a framework — no heavy dependency.

Caveat to state honestly: OKF is v0.1 and new. Treat the format as convention, not as a dependency — write your own loader (~80 lines) rather than adopting an SDK, so a spec change can't break you.

What OKF is not: a metric-computation engine. For that, the real options are **Malloy** (Google's actual open-source data modeling language, created by Looker's founder), **dbt MetricFlow** (open-sourced late 2025, Apache 2.0), or **Cube Core** (Apache 2.0). **My recommendation: don't adopt any of them yet.** Your metric definitions are ~20 entries; a YAML file plus a small resolver beats a framework. Note Malloy/MetricFlow in `PROVENANCE.md` as the migration path if metric complexity grows — that's the mature answer to "why didn't you use a semantic layer?"

Rule Book structure — clustered, as you asked

Your instinct about maintaining "clusters of OKFs" is right. Organise by **bundle**, where a bundle is a coherent domain:

☐

rulebook/

└─ okf.yaml # bundle manifest, version, owners

└─ metrics/

│ └─ geology/

│ │ └─ seam_thickness_gross.md

│ │ └─ seam_thickness_net.md

│ │ └─ coal_ash_pct.md

│ │ └─ coal_gcv.md

│ │ └─ coal_grade_band.md

│ └─ production/

│ │ └─ coal_production_raw.md

│ │ └─ overburden_removal.md

│ └─ resource/

│ │ └─ resource_proved_mt.md

│ │ └─ unfc_classification.md

└─ environment/

│ └─ land_reclaimed_technical.md

└─ land_reclaimed_biological.md

└─ entities/

│ └─ subsidiaries.md

│ └─ coalfields.md

└─ borehole_series.md

└─ authority/

└─ source_ranking.md

└─ rules/ # the YAML consumed by Phase 4

│ └─ geology.yaml

└─ production.yaml

└─ conformance/

└─ nmet_gr_g2.yaml

A real Rule Book entry:

☐
markdown

```
---
okf_version: "0.1"
id: seam_thickness_gross
kind: metric
version: 3
canonical_unit: m
value_type: decimal
metric_class: length
domain: geology
owner: "GM (Geology), CMPDI"
authority_sources: [CMPDI, GSI, MECL]
non_substitutable_with: [seam_thickness_net]
qualifiers_required: [seam, borehole]
aggregation: none      # thickness is never summed across boreholes
allowed_scales: null
review_tier: geologist
supersedes_version: 2
changed_on: 2026-08-14
change_reason: "Added qualifiers_required after Marwatola ingest showed thickness
                is meaningless without both seam and borehole context"
---
```

Gross Seam Thickness

The full vertical thickness of a coal seam as intersected in a borehole,
including dirt bands and stone partings within the seam.

Not to be confused with
Net thickness — which excludes dirt bands and partings. Gross and net
thickness are separate metrics and **must never be substituted for one another**
in a resource statement. Net thickness is always ≤ gross thickness for the same

seam and borehole; a violation indicates an extraction error.

Measurement

Recorded in metres from the borehole log, as the difference between the seam roof and floor depths. Reported to two decimal places in CMPDI Geological Reports.

Reporting caution

Gross thickness alone does not determine mineable thickness. A seam with high gross thickness but thick internal partings may have substantially lower net and workable thickness.

Provenance of this definition

Derived from CMPDI Geological Report conventions as observed in the Marwatola Sector I & II (G2) report, Sohagpur Coalfield, and the NMET mandated GR submission format for G4/G3/G2 stages.

Read that entry and notice what it gives you. A validation rule (net ≤ gross), a non-substitution constraint, a required-qualifier set, a review tier, a reporting caution the Narrative Track can surface, a named owner, and a change reason. **All of that is now enforced by code and readable by a geologist.** This single file format is what makes your "institutional memory becomes an asset" claim real.

Modules

M6.1 OKF Loader — parse frontmatter + body, validate against a Pydantic model, ~80 lines.

M6.2 Compiler — OKF files → Postgres `metric_def`, `authority_rank`, `alias` tables. Runs in CI on commit to `rulebook/`.

M6.3 Version Registry — every Assay run and every fact records `rulebook_version`. Non-negotiable for reproducibility.

M6.4 Diff & Impact — on a Rule Book change, report which published facts were validated under the old version and might now be affected. Feeds re-evaluation.

M6.5 Rule Book Browser — a read-only UI rendering the markdown, with "which facts use this metric" and "which rules derive from this."

Best practices & my additions (8)

Rule Book is Git-primary, Postgres-derived. Never edit the projection directly; it's a build artifact.

Every fact records the Rule Book version that validated it. Without this, reproducing an old answer is impossible.

`change_reason` **is mandatory in frontmatter.** Enforce it in CI. Six months on, this file is the only record of *why* a definition is what it is.

Compile in CI, fail the build on invalid OKF. A malformed Rule Book must never reach the runtime.

My addition — bundle-level owners. Each cluster names an accountable role (geology → GM Geology). This mirrors real ministry accountability and is a credible detail on stage.

My addition — a `rulebook diff --impact` command. Bump `seam_thickness_gross` to v4 and see: 418 published facts validated under v3, 12 of which would now fail. That's your "definition drift" story, working.

My addition — render the Rule Book into your UI. A geologist judge reading a real definition page written in their vocabulary is worth more than any diagram.

My addition — seed it from the NMET GR format document. Your definitions then trace to a government standard, not to your judgment. Say that: *"we didn't design a schema for geological reports; we implemented the Ministry's published submission format."*

How to measure

Rule Book coverage (% of metric keys in use that have a definition — target 100%, enforce in CI). Time to add a new metric end to end (target < 30 minutes, no code change). Number of facts whose validating version is recoverable (must be 100%).

PHASE 6.5 — TOPIC INTELLIGENCE PREREQUISITES

(You correctly spotted this gap. Here is what to add *now*, cheaply.)

Why this belongs in the data layer

Topic Intelligence is a *later* service, but it needs things that are **expensive to add retroactively and nearly free to add now**. If you skip these, you'll re-ingest your whole corpus later.

What Topic Intelligence is actually for (in coal terms)

Not a word cloud. Four purposes, and two of them point inward:

Lens	Coal-domain purpose
What is discussed	Which technical themes dominate the archive — distinctive terms, not "coal, mine, report"
What is rising	Is "coal gasification" or "biologically reclaimed area" accelerating in questions before it accelerates in our documents?
What cannot be proved	Which (metric, entity, period) combinations are asked about but unevidenced → digitisation work orders
What contradicts itself	Where our own published statements disagree

The seven things to add now (each is small)

1. **Time-bin every chunk and document.** You have `published_on`. Add a derived `time_bin` (quarter). Burst detection needs a time series; adding dates later means re-processing.

2. **Store per-chunk term statistics at ingest.**

☐

sql

```
CREATE TABLE chunk_term (  
  chunk_id text, term text, tf int, is_phrase bool, is_entity bool,  
  PRIMARY KEY (chunk_id, term)  
);  
  
CREATE TABLE corpus_term_stats (  
  term text PRIMARY KEY, df int, total_tf int,  
  authority_weighted_tf real, --  $\sum \text{tf}/\text{authority\_rank}$   
  first_seen date, last_seen date  
);
```

Computing TF-IDF later from raw text means re-tokenising everything. Computing it from these tables is a SQL query. **This is the single cheapest high-value addition in this phase.**

3. Persist the domain label array on documents. You already set `domain: ['geology','resource']` in the graph. Make sure it's on the `document` row too, for filtering.

4. Ingest the demand side. Parliamentary questions and administrative queries are a *corpus*, not an afterthought. Same chunking pipeline, `doc_kind='pq_reply'`. **Without a demand corpus, your evidence-gap lens cannot exist.** This is why Wave 2 in Phase 1 includes Q&A PDFs.

5. A `coverage` materialised view — the substrate of the gap lens:

```
sql  
  
CREATE MATERIALIZED VIEW coverage AS  
SELECT metric_key, entity_id, period,  
  count(*) FILTER (WHERE system_to IS NULL) AS fact_count,  
  max(status) AS best_status  
FROM fact GROUP BY 1,2,3;
```

Join demand against this and gaps fall out. Two tables, one join, your novelty.

6. A `published_statement` table — the substrate of the contradiction lens:



sql

```
CREATE TABLE published_statement (  
  stmt_id text PRIMARY KEY,  
  doc_id text NOT NULL, doc_kind text NOT NULL,  
  published_on date, approver text,  
  metric_key text, entity_id text, period text,  
  value numeric, unit text,  
  rulebook_version int,  
  used_in_pq bool DEFAULT false, -- exposure flag  
  source_ref jsonb  
);
```

Every statement your organisation has *published* about a number. Group by (metric, entity, period), find groups with divergent values → contradictions. **Populate this from your ingested annual reports and PQ replies as you go.** Retroactively extracting it is a project; doing it during ingest is a table insert.

7. A controlled topic taxonomy stub in the Rule Book. `rulebook/topics/taxonomy.md` with 15–20 approved coal topics (Mine Closure, Exploratory Drilling, Environmental Clearance, Production Target, Seam Thickness, Coal Quality, Land Reclamation, Water Management, Subsidence, Rehabilitation...). Discovered topics later get promoted *into* this file by a reviewer — which is your topic-governance story with a real artifact behind it.

My additions (5)

Store both raw and contextualised text per chunk. Topic modelling should run on *raw* text (context prefixes would pollute term statistics with repeated boilerplate). You need both. This is a subtle trap worth avoiding.

Record a stopwords list per domain, in the Rule Book. In coal documents, "coal", "mine", "seam", "report", "block" are effectively stopwords for distinctiveness purposes. Making this a versioned Rule Book artifact rather than a hardcoded list is both better engineering and a nice detail.

Tag chunks with `state` (from block → district → state). Geographic concentration of topics is one of your four lenses and needs this join to exist.

Count entity mentions per document at ingest. Entity clouds are more useful than term clouds in this domain, and the counts are free during Phase 3.

Log every unanswerable query the system encounters, from day one. Even in MVP-1. This becomes your organic demand corpus, and by MVP-4 you'll have real gap data rather than simulated data. **Nobody else will think to do this.**

PHASE 7 — BEDROCK (the Evidence Broker)

The name

BEDROCK — *Broker for Evidence, Documents, Relations, Ontology, Coverage & Knowledge.*

It's a genuine geological term (the solid rock beneath loose material), it's literally the foundation layer of your architecture, the acronym is honest about what it serves, and it will land with a CMPDI audience. Your slide line: **"BEDROCK is the brain. The unified layer is the heart."**

Layman

One office, one door. Every AI service — a query assistant, a report generator, a parliamentary-reply drafter — asks BEDROCK for evidence rather than going to the databases itself. BEDROCK checks who's asking, gathers only what they're allowed to see, seals it with a serial number, and hands it over. Nobody downstream can disagree with anybody else, because everyone got the same sealed folder.

Techy

A role-aware MCP server exposing Resources, Tools and Prompts over the unified knowledge layer, returning immutable content-addressed Evidence Packages, with tiered service guarantees per consumer class.

7.1 Is MCP the right choice? Honest analysis

MCP's real advantage for you: your three consumers are all LLM-based agents. MCP is designed so an agent can *discover* capabilities and use them without bespoke integration code per service. That's genuinely valuable, and it's architecturally current.

MCP’s real limitation, which you must know: the MCP specification (the 2025-06-18 revision added OAuth 2.1) covers *transport authentication*. **Role-aware conditional exposure of tools — showing different tools to different callers — is NOT natively specified. It is an application-layer concern you must implement.** Multiple 2026 practitioner sources stress designing per-tool scoping in from the start rather than retrofitting middleware.

The alternatives, compared honestly:

Option	Ease for your team	Security maturity	AI-native	Verdict
MCP server	Medium (SDK is good)	Low — you carry the burden	High	✔ Use it, but keep auth in a tested app layer
Plain REST + OAuth	High	High — battle-tested	Low (agents need wrappers)	Good fallback; less impressive
GraphQL + field auth	Medium-low	Medium-high	Medium	Over-complex for your scale
gRPC	Low	High	Low	No

My recommendation: build BEDROCK as an MCP server, but treat MCP as the *interface*, not the *security boundary*. Authorization lives in a separate, tested module that both `tools/list` and `tools/call` consult. Say exactly that on stage — it shows you know where the spec ends.

Reference patterns exist: InfraCloud published a Keycloak+OAuth RBAC MCP server demonstrating per-tool RBAC with viewer/user/admin roles and a `get_my_permissions` meta-tool. CodiLime’s network-automation series shows JWT auth with scope-based per-tool authorization. Cloudflare’s MCP portal pattern uses `default_disabled` plus explicit allow-lists for role-based tool hiding. **Study the InfraCloud one — it’s closest to what you need.**

7.2 Role-aware tool exposure — the implementation

Enforce at both layers. This is the critical detail:

```


python


```

```
# src/bhumi/broker/authz.py
```

```
@dataclass(frozen=True)
```

```
class Principal:
```

```
    subject: str
```

```
    role: Literal["parliament_section", "cmpdi_geologist", "subsidiary_officer",  
                 "auditor", "public"]
```

```
    org: Optional[str]          # "SECL" — drives ReBAC scoping
```

```
    purpose: Literal["parliamentary_answer", "technical_review",  
                    "internal_report", "public_disclosure"]
```

```
    max_classification: Literal["public", "internal", "restricted"]
```

```
    scopes: frozenset[str]
```

```
# Declarative policy — data, not code
```

```
TOOL_POLICY = {
```

```
    "search_evidence": {"scopes": {"evidence:read"}},
```

```
    "get_fact": {"scopes": {"fact:read"}},
```

```
    "compute_metric": {"scopes": {"fact:compute"}},
```

```
    "traverse_graph": {"scopes": {"graph:read"}},
```

```
    "get_provenance": {"scopes": {"lineage:read"}},
```

```
    "check_coverage": {"scopes": {"coverage:read"}},
```

```
    "seal_evidence_package": {"scopes": {"evidence:seal"}},
```

```
    "list_geological_tables": {"scopes": {"geology:read"},  
                              "min_classification": "restricted"},
```

```
    "get_conformance_report": {"scopes": {"geology:review"}},
```

```
    "list_review_queue": {"scopes": {"assay:review"}},
```

```
}
```

```
ROLE_SCOPES = {
```

```
    "parliament_section": {"evidence:read", "fact:read", "fact:compute",
```

```
                           "graph:read", "lineage:read", "coverage:read", "evidence:seal"},
```

```

"cmpdi_geologist": {"evidence:read","fact:read","fact:compute","graph:read",
                    "lineage:read","geology:read","geology:review",
                    "assay:review","evidence:seal"},
"subsidiary_officer": {"evidence:read","fact:read","fact:compute",
                       "graph:read","lineage:read","coverage:read","evidence:seal"},
"auditor":          {"evidence:read","fact:read","lineage:read"}, # read-only
"public":           {"evidence:read","fact:read"},
}

```

```

def visible_tools(p: Principal) -> list[str]:
    """Called by tools/list — VISIBILITY."""
    return [t for t, pol in TOOL_POLICY.items()
            if pol["scopes"] <= p.scopes
            and _class_ok(pol.get("min_classification"), p.max_classification)]

def authorize_call(p: Principal, tool: str, args: dict) -> None:
    """Called by tools/call — EXECUTION. Must be independent of visibility."""
    if tool not in visible_tools(p):
        audit.deny(p, tool, "not_visible")
        raise PermissionError(f"tool '{tool}' not available for role {p.role}")
    _check_row_scope(p, tool, args) # ReBAC: SECL officer → SECL data only
    audit.allow(p, tool, args)

```

Why both layers, independently: hiding a tool is UX; blocking the call is security. A caller who guesses a tool name must still be refused. Testing them separately is a genuine security property.

What this looks like in practice — three personas, one server:



PARLIAMENT SECTION sees 7 tools:

search_evidence · get_fact · compute_metric · traverse_graph
get_provenance · check_coverage · seal_evidence_package

CMPDI GEOLOGIST sees 9 tools (+geology, +review, -coverage):

... + list_geological_tables · get_conformance_report · list_review_queue

AUDITOR sees 3 tools (read-only, no compute, no seal):

search_evidence · get_fact · get_provenance

That is your "same question, two officers, two correct answers" demo — implemented, not claimed. Switch the token, watch the tool list change, ask the same question, get a different evidence set.

7.3 The Evidence Package (the sealed artifact)



python

```

class EvidencePackage(BaseModel):
    package_id: str      # "SEP-" + sha256(canonical_json)[:12]
    content_hash: str     # full sha256 — printed in report footers
    created_at: datetime

    intent: Intent        # parsed entities, metrics, periods, purpose
    policy_snapshot: PolicySnapshot # role, purpose, max_classification, scopes
                                # ← policy is IN the package. cache key too.
    facts: list[Fact]
    passages: list[Passage]    # chunk + parent + bbox
    relations: list[Relation]   # with trust_layer
    rules: list[RuleBookEntry]  # definitions actually invoked
    conflicts: list[Conflict]
    coverage: Coverage          # answerable / partial / absent per ask
    cutoff: date
    route_trace: list[RouteStep] # which store answered what, in order

    rulebook_version: int
    replayable: bool = True

```

Five properties this one artifact buys you — say all five, they compound:

Consistency. A chat answer and an official report cannot disagree; both render from one package ID.

Auditability. The hash goes in the footer. An auditor demands package `SEP-A7F3C1` and gets exactly what was known and permitted at that moment.

Cacheability. Cache key = hash(intent + policy_snapshot). **Policy must be in the key** — otherwise you leak restricted evidence through the cache to a lower-privilege caller. This is the single most likely security bug in the whole system; naming it shows you've thought about it.

Testability. Golden set = (intent → expected package). You benchmark *retrieval* separately from *generation*.

Security correctness. Policy applied once, at seal time. One place for a permission bug to live.

7.4 The MCP surface — Tools, Resources, Prompts

Tools (verbs the agent calls):

Tool	Input	Output	Backed by
<code>search_evidence</code>	query, filters, k	ranked passages w/ bbox	hybrid RRF + parent expand
<code>get_fact</code>	fact_identity (entity, metric, period), as_of?	Fact + provenance	Fact Ledger, bitemporal
<code>compute_metric</code>	metric_key, entity, period, qualifiers	value + expression tree + input fact IDs	Rule Book + governed SQL
<code>traverse_graph</code>	start, pattern, max_hops, min_trust	subgraph	Apache AGE Cypher
<code>get_provenance</code>	fact_id claim_id	full lineage chain + bbox	lineage table, backward
<code>check_coverage</code>	list of asks	covered/partial/absent + owner + gap code	coverage matview
<code>list_geological_tables</code>	block coalfield seam	typed tables + conformance	domain typing + AST
<code>get_conformance_report</code>	doc_id	mandatory-section checklist	conformance rules
<code>seal_evidence_package</code>	intent	EvidencePackage	the whole layer
<code>whoami</code>	—	role, purpose, scopes, visible tools	authz

Resources (nouns the agent reads — this is what makes it self-describing):

Resource URI	Content
<code>bedrock://rulebook/metrics</code>	all metric definitions the caller may see
<code>bedrock://rulebook/metric/{key}</code>	one full OKF definition, rendered
<code>bedrock://schema/entities</code>	entity types + counts + example IDs
<code>bedrock://schema/graph</code>	node labels, edge types, trust layers
<code>bedrock://corpus/manifest</code>	documents available to this caller
<code>bedrock://coverage/summary</code>	what can and cannot be proved
<code>bedrock://policy/mine</code>	this caller's own permissions, explicitly

`bedrock://schema/*` **is the highest-leverage design choice here.** An agent that can read the graph schema and the metric catalogue writes correct queries on the first try instead of guessing. This is what "easy for agents to speak up" actually requires.

Prompts (reusable templates BEDROCK offers its clients):

Prompt	Purpose
<code>decompose_question</code>	one question → atomic asks bound to metric keys
<code>plan_evidence</code>	asks → a retrieval plan (which tools, what order)
<code>ground_narrative</code>	package → prose citing figure IDs only
<code>explain_gap</code>	gap code → officer-readable explanation with owner and SLA

7.5 Service tiers — your Spotify/YouTube insight, formalised

Your analogy is exactly right, and it has proper systems-design names: **graceful degradation**, **cache-then-serve**, **circuit breaker**, and **strong versus eventual consistency**. Cite those terms — they're the authoritative vocabulary.

Three tiers, declared per caller:

	Tier S — Statutory	Tier O — Operational	Tier E — Exploratory
Consumer	PQ Desk, Report Engine	CIL/CMPDI officer queries	Topic Radar, browsing
Analogy	YouTube — quality or nothing	mixed	Spotify — always works
Consistency	Strong, current-only	Read-committed	Eventual, cache-first
Cache	Never serve stale	≤5 min	≤24h, serve stale gladly
On store failure	FAIL LOUDLY. Never partial	Degrade + flag	Serve cached silently
Provisional facts	Flagged, footnote forced	Flagged	Included
Derived graph edges	Excluded	Excluded from numerics	Included
Latency budget	30s–5min (correctness > speed)	<3s	best-effort
Audit log	Every access, full package retained	Sampled	Aggregate only

The rule that matters: *"For a statutory answer, a partial result is worse than no result. For an exploratory query, a stale result is better than an error."*



python

```

class ServiceTier(str, Enum):
    STATUTORY = "S"; OPERATIONAL = "O"; EXPLORATORY = "E"

TIER_POLICY = {
    ServiceTier.STATUTORY: dict(max_cache_age=None, allow_partial=False,
                                allow_stale=False, include_derived_edges=False,
                                require_current=True, audit="full"),
    ServiceTier.OPERATIONAL: dict(max_cache_age=300, allow_partial=True,
                                   allow_stale=False, include_derived_edges=False,
                                   require_current=True, audit="sampled"),
    ServiceTier.EXPLORATORY: dict(max_cache_age=86400, allow_partial=True,
                                   allow_stale=True, include_derived_edges=True,
                                   require_current=False, audit="aggregate"),
}

```

This table is a slide. Non-functional requirements, declared per consumer, with a defensible reason for each cell. Very few student teams will have thought about this at all.

7.6 Your Report Engine question: does template + metadata → plan work?

Yes, it works — and it's the right instinct. Three refinements make it much stronger.

Refinement 1 — the plan must be a declarative artifact, not a prompt.

 python

```
class ReportPlan(BaseModel):
    template_id: str
    sections: list[SectionPlan] # each: required metrics, table?, chart?, narrative allowed?
    approver_rank: str
    disclosure_obligations: list[str] # cutoff, hindi, mixed-status footnote
    plan_hash: str # so the plan itself is auditable
```

An LLM may *propose* a plan; the plan is then **validated against the Rule Book** (do these metric keys exist? is this aggregation legal?) before execution. A hallucinated section can't reach the pipeline.

Refinement 2 — coverage check between plan and drafting. This is the key ordering:

```
□
template + metadata → PLAN → validate plan against Rule Book
→ seal package per section → CHECK COVERAGE
→ gaps declared and routed BEFORE a word is drafted
→ compute → narrate → gate → approve
```

Coverage before drafting is what makes the Report Engine honest rather than fluent. It's also cheap: one join against the `coverage` matview.

Refinement 3 — BEDROCK serves the loop, not just one call. A report is iterative: plan → discover a gap → re-plan → compute → verify → revise. So BEDROCK needs to support **conversational, stateful evidence gathering**:

`seal_evidence_package` accepts an optional `parent_package_id` → creates a *lineage of packages*, so you can see how evidence accumulated across the report's construction.

Packages are **immutable but composable**: `merge_packages([ids]) → new sealed package`, with the parent IDs recorded.

The final report footer carries the **root package hash**, and the chain of parents is auditable.

That last point is a genuinely nice piece of design: a report's evidence has a *history*, not just a snapshot.

7.7 Concrete flow — end to end



— CLIENT: PQ Desk agent, MCP client —

initialize(token=eyJ...)

→ BEDROCK resolves Principal{role=parliament_section, purpose=parliamentary_answer,
max_classification=public, tier=STATUTORY}

tools/list → 7 tools (list_geological_tables NOT shown — restricted)

resources/read bedrock://rulebook/metrics → 23 metric definitions

resources/read bedrock://schema/graph → node labels, edge types, trust layers

prompts/get decompose_question

question: "gross thickness of Seam III Top in Marwatola Sector I & II"

→ asks: [{metric: seam_thickness_gross,
qualifiers: {seam: SEAM:MARWATOLA:III_TOP, block: BLK:MARWATOLA_I_II},
period: latest}]

tools/call check_coverage(asks)

→ A1: COVERED — 3 facts (3 boreholes intersect this seam)

tools/call seal_evidence_package(intent)

→ SEP-A7F3C1E90B24

content_hash 7f3c1e90b24a8f... (footer-printable)

facts 3:

F-000418 BH:SKM-12 3.42 m final conf 0.90 p214 Table 6.2 r7c2

F-000419 BH:SKM-14 3.38 m final conf 0.94 p214 Table 6.2 r9c2

F-000431 BH:SGT-07 3.51 m final conf 0.88 p215 Table 6.2 r22c2

passages 4 (Table 6.2 parent + 3 contextualised rows)

relations 6 (INTERSECTS ×3, IN_BLOCK, IN_COALFIELD, DESCRIBES)

all trust_layer ∈ {authoritative, validated} — derived EXCLUDED (Tier S)

rules 2 seam_thickness_gross v3, seam_thickness_net v3

conflicts 0

coverage answerable, no gaps

```
cutoff 2019-03-31 (report date; latest validated for this block)
route_trace [rulebook→bind, ledger→facts, index→passages, graph→relations]

tools/call get_provenance(F-000418)
→ claim → fact F-000418 → assay_run_2026-08-14T09:12 (7 gates, 1 warn)
→ candidate cand_8f21a04c → cell r7c2 page 214 bbox(186.0,478.3,242.1,490.1)
→ element #/tables/17 → GR-MARWATOLA-I-II-G2 v1
→ artifact 4f9a2c81e7... (WORM, sha256 verified)
→ approved_by geologist:reviewer_02 on 2026-08-14T11:40

— the agent now drafts, citing F-000418 by ID, never typing "3.42" —
```

```
— SAME QUESTION, DIFFERENT PRINCIPAL —
initialize(token=<cmpdi_geologist>, purpose=technical_review,
    max_classification=restricted, tier=OPERATIONAL)
tools/list → 9 tools (list_geological_tables NOW VISIBLE)
tools/call seal_evidence_package(same intent)
→ SEP-D22B8F0164AA ← DIFFERENT PACKAGE, DIFFERENT HASH
facts 5 (+2 from a restricted annexure the Parliament persona cannot see)
relations 11 (derived similarity edges included — Tier O for non-numeric use)
conformance_report available
```

That side-by-side is your single best demo of the whole data layer. Same question. Different principal. Different tool list, different evidence, different package hash — and both are correct.

Best practices & my additions (10)

Enforce authorization at `tools/list` AND `tools/call`, independently. Visibility is UX; blocking is security.

Policy snapshot inside the package, and inside the cache key. The cache is the most likely place to leak restricted evidence. Write a test for exactly this.

Content-address the package. The hash *is* the identity. Deterministic JSON canonicalisation (sorted keys, fixed float format) or your hashes won't reproduce — this trips up nearly everyone the first time.

Never let a consumer bypass BEDROCK. No direct DB access from any service. Enforce with separate DB roles: services get *no* table grants, only BEDROCK does.

`whoami` and `bedrock://policy/mine` **as first-class.** An agent that can inspect its own permissions writes better queries and fails more gracefully. Also a great demo.

My addition — `bedrock://schema/*` resources. Self-describing schema is what makes agents effective. Highest-value resource you can expose.

My addition — `route_trace` in every package. Which store answered what, in order. This is your Route Board UI, and it's how you debug retrieval.

My addition — declare the tier per caller and log it. Then you can prove that a statutory answer never used a cached or partial result. That's an auditable non-functional guarantee, which is rare and impressive.

My addition — package composition with parent lineage. Reports gather evidence iteratively; a package that records its parents makes that history auditable.

My addition — a `replay(package_id)` command. Re-run a sealed package's intent against current data and diff. This is the seed of your Revision Impact Trace, and it's a five-minute demo that lands hard.

How to measure Phase 7

Metric	How
Cross-role leakage	Red-team set: for each restricted fact, assert no lower-privilege principal can reach it via any tool. Must be zero. Make it a CI test.
Package determinism	Same intent + policy → identical hash, 100%
Tool visibility correctness	Assert exact expected tool list per role
p95 seal latency	Cold vs cached, by tier
Cache hit rate + zero cross-policy hits	The second is a security assertion, not a performance one

Metric	How
Tier compliance	Assert no STATUTORY package ever contained stale/partial/derived content
Agent first-try success	With vs without <code>bedrock://schema/*</code> exposed. Quantifies your design choice

File structure after Phase 7



```
src/bhumi/broker/
├── server.py      # MCP server entrypoint
├── authz.py      # Principal, TOOL_POLICY, ROLE_SCOPES, both gates
├── tiers.py      # ServiceTier + TIER_POLICY
├── package.py    # EvidencePackage, canonical hash, compose, replay
├── tools/*.py    # one file per tool
├── resources/*.py # schema, rulebook, corpus, coverage, policy
├── prompts/*.py  # decompose, plan, ground, explain_gap
├── cache.py      # policy-aware cache
├── audit.py      # every allow/deny, structured
├── tests/broker/test_no_cross_role_leakage.py ← the most important test in the repo
└── ui/pages/7_bedrock_console.py             ← the three-persona demo screen
```

✓ **MVP-4 IS LIVE HERE — this is your submission**

Three personas, one server, visibly different tool lists and evidence. Every figure drills to a source cell. Rule Book edits change system behaviour live. Pipeline trace proves nothing is mocked.

PART 8 — THE MVP LADDER, CONSOLIDATED

MVP Phases		Working application	Video-able moment
0	0	<code>make verify</code> green	—
1	+1,2	Real GR → document tree → click cell → source page with rectangle	"Click any number, land on the cell"
2	+3,4	Domain-typed tables → candidates → decomposed confidence → review queue → published facts	"Seven gates. Watch one fail, and watch it recover after a rule change"
3	+5,6.5	Hybrid search, knowledge graph, 5-hop query, pipeline trace, bitemporal as-of	"Five hops. No vector search answers this"
4	+6,7	Rule Book live, BEDROCK three personas, sealed packages, replay	"Same question, two officers, two correct answers, two hashes"

Where the \$1,500 Claude Code budget goes

Use	Share	Why
Code generation across all phases	~55%	The bulk. Claude Code writes most of this codebase
Golden-set assistance (labelling help, ground-truth HTML for tables)	~15%	Accelerates your most valuable manual asset
Domain-pack authoring from real GRs	~10%	Reading reports and drafting YAML
Entity-resolution adjudication on hard pairs	~8%	Research-supported use of an LLM
Contextualisation prompt development + one-off batch runs	~7%	Then switch to the local 3B for bulk
Rule Book drafting from the NMET format	~5%	High-value, low-token

Development uses Claude generously. Production inference must be local — that’s your cost story for the ministry, and it’s real.

PART 9 — WHAT TO LEARN, AND WHEN

Sequenced to when you'll actually need it. Learning ahead of need doesn't stick.

Before	Learn	Why
Phase 0	<code>uv</code> , Docker Compose, Alembic basics, Pydantic v2	Foundation; a schema mistake here is expensive
Phase 2	PDF internals (text layer vs image layer); read Docling's docs <i>properly</i> , once	Saves a week of confused debugging
Phase 2	GGUF quantisation levels and llama.cpp basics	Your 6GB constraint makes this non-optional
Phase 3	spaCy <code>EntityRuler</code> ; regex discipline	Deterministic NER without training data
Phase 3	Fellegi-Sunter intuition (not the maths)	So you can explain Splink's scores to a judge
Phase 4	Confidence calibration; conformal prediction	Turns a number into a guarantee
Phase 4	Writing a good unit test for a validation rule	The single highest-leverage habit for this project
Phase 5	Postgres: CTEs, window functions, GIN/HNSW indexes, <code>EXPLAIN ANALYZE</code>	Your whole knowledge layer is SQL
Phase 5	openCypher basics + Apache AGE's quirks	Graph queries
Phase 5	Deterministic JSON canonicalisation	Or your package hashes won't reproduce
Phase 7	The MCP spec, read directly — not a blog summary	You're implementing the security the spec doesn't
Phase 7	Graceful degradation, circuit breaker, cache-aside	The vocabulary for your tier design
Throughout	Recall@k, MRR, nDCG, TEDS	You can't choose techniques you can't measure

PART 10 — WHERE TO INNOVATE (the part I cannot do for you)

Everything above is engineering — real, necessary, buildable by following instructions carefully. These five are yours.

- 1. Read the Marwatola GR slowly, with a domain person nearby.** I found the document and told you its structure. I have not sat with it for an hour asking "what would a geologist consider obvious that an outsider would get wrong?" **That is the single highest-value hour available to you on this entire project** — not coding, reading, with someone who knows coal, asking "what did I just misunderstand?" Every genuine domain novelty you produce will trace back to that hour.
- 2. The severity calibration in your rule set is a judgment call with no textbook answer.** Is `ash_gcv_inverse` a warning or a block? At what divergence does a cross-document difference become a conflict rather than rounding? What materiality tolerance is right? I can give you the mechanism; only you can defend the threshold. And being able to explain *why* you chose 0.6% rather than 5% is exactly the kind of thinking a domain judge respects — because it can't be looked up.
- 3. The golden dataset can only be built by hand, and it is your only truly unique asset.** Twenty-five hand-labelled tables. Sixty questions with verified answers and exact supporting evidence. Forty sentences, half deliberately unsupported. It is tedious and unglamorous, and **no other team will do it**, which is precisely why it's worth more than any feature.
- 4. Watch for the moment real data does something weirder than my clean examples.** Every strong novelty in this design came from combining a domain fact with a technical pattern. The next one — genuinely yours — will come the same way, from something you notice while building Phase 2 or 4 that I cannot predict from here. **When real data does something strange, stop and ask "is this weirdness itself a feature?"** rather than handling it and moving on. Almost every real novelty starts as an annoying edge case someone nearly brushed past.
- 5. Don't let the pitch outrun the build.** This document describes more than you can finish. That's fine and normal. The discipline that separates a winning team from an impressive-sounding one is refusing to claim, out loud, anything your code doesn't do yet. Every honest "that's designed but not built" is a credibility asset. Every implied capability that turns out to be a mock when a judge pokes it costs you the room.

PART 11 — VERIFY THESE BEFORE YOU QUOTE THEM

The 1 January 2012 UHV→GCV grading switchover date. I'm confident, but you'll be saying it to people who lived through it. Confirm on coal.gov.in or the Coal Controller's site.

All NMET and CCO download URLs. Government sites reorganise. Vault everything locally today.

Whether the Marwatola GRs contain full borehole log tables or only summaries. Open the PDFs and check before designing your extraction schema around them.

Bulk parliamentary Q&A access. Coal Q&A is published on sansad.in and eparlib.nic.in, but I could not verify a bulk download or API.

The 1,250-GR figure is from CMPDI's 2006-07 annual report and has almost certainly grown. Always say "as reported in 2006-07."

Kuzu's project health — the original repo was archived in October 2025, now maintained as a community fork. Verify before adopting; Apache AGE is the lower-risk choice.

Oracle Cloud Always Free limits were cut mid-2026 (reportedly to ~2 OCPU/12GB from 4/24GB). Re-verify before planning deployment around it.

Vendor-reported OCR benchmarks run above independent leaderboards. Reproduce PaddleOCR-VL's numbers on *your* documents before quoting anything.

Marker's licence (GPL-3.0 + RAIL-M revenue restriction) — I'd avoid it entirely for a government deployment.

Next iteration: the Report Engine, Query Engine and Topic Radar as consumers of BEDROCK. Send me Phase 2's actual output on a real Marwatola page first — that's the checkpoint where this plan either holds or needs revising against evidence.